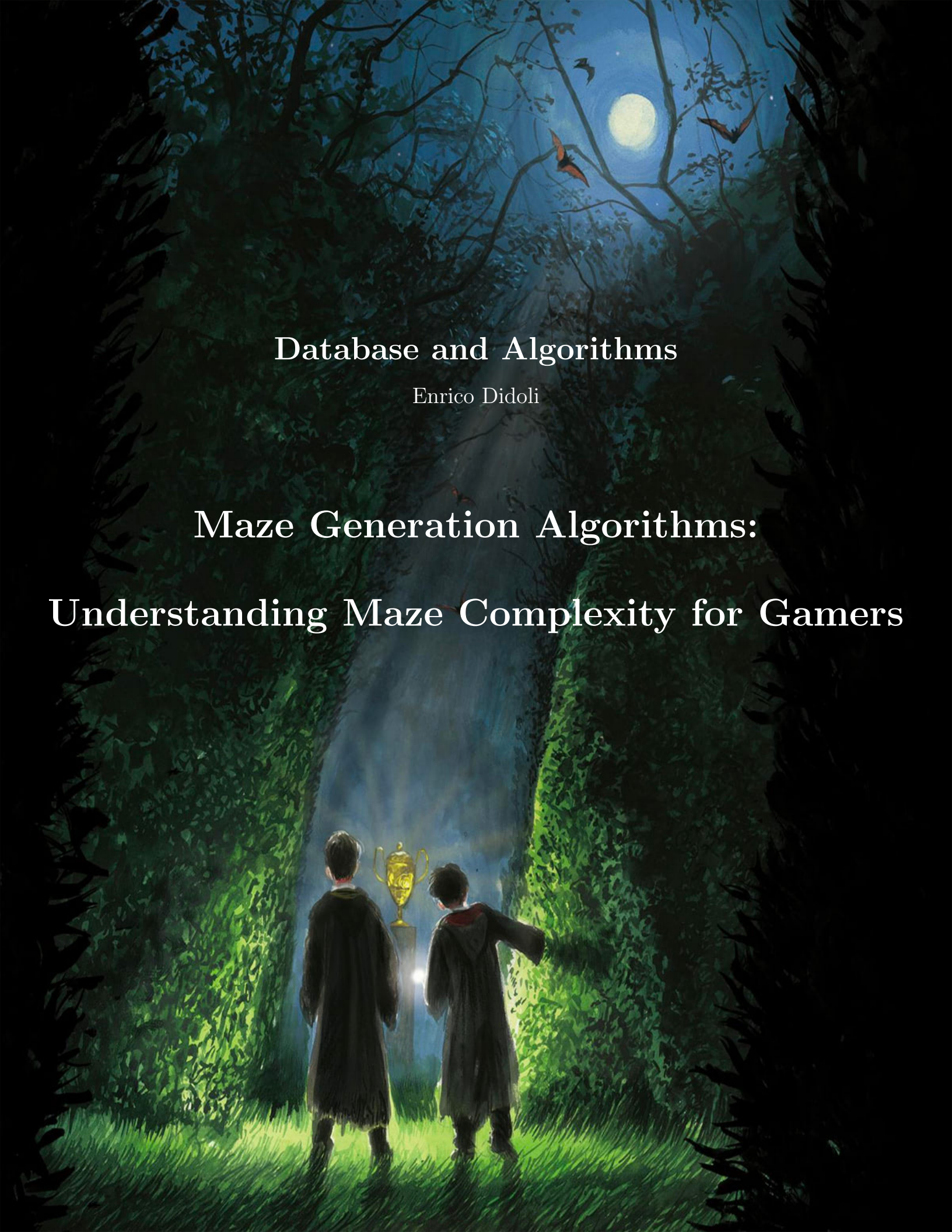
Database and Algorithms

Enrico Didoli

Maze Generation Algorithms:

Understanding Maze Complexity for Gamers



Contents

1 Introduction **4**
- 1.1 What is a maze? . . . 4
- 1.2 Objectives of the Study . . . 4

2 Mathematical Modeling **5**
- 2.1 Graph Theory . . . 5
- 2.2 Random Walks . . . 7
- 2.3 Metrics . . . 8
  - 2.3.1 Maze Inherent Metrics . . . 8
  - 2.3.2 Solver-Based Metrics . . . 8

3 Algorithms Presentation **10**
- 3.1 Aldous-Broder . . . 10
- 3.2 Wilson . . . 11
- 3.3 Randomized Prim's Algorithm . . . 13
- 3.4 Recursive Backtracking (Depth-First Search) . . . 15
- 3.5 Eller's Algorithm . . . 17

4 A simple Maze Generation **19**
- 4.1 Step by step generation . . . 19

5 Time and Space Complexity Analysis **22**
- 5.1 Time Complexity Analysis . . . 22
  - 5.1.1 Aldous-Broder . . . 23
  - 5.1.2 Wilson's Algorithm . . . 23
  - 5.1.3 Randomized Prim . . . 24
  - 5.1.4 Backtracking Algorithm . . . 25
  - 5.1.5 Eller . . . 26
- 5.2 Space Complexity Analysis . . . 28

6 Comprehensive Analysis: Maze Generation Algorithms and A* Performance **30**
- 6.1 Maze Inherent Metrics Analysis . . . 30
  - 6.1.1 Raw Data Summary . . . 30
  - 6.1.2 Generation Analysis . . . 31
- 6.2 Solver-Based Metrics Analysis . . . 32
  - 6.2.1 Raw Data Summary . . . 32
  - 6.2.2 Performance Analysis . . . 33

7 Conclusion **39**
- 7.1 Future Studies . . . 39

8 Appendix A **40**
- 8.1 How structure mazes as Trees? . . . 40

9 Appendix B **43**
- 9.1 Aldous-Broder . . . 43
- 9.2 Wilson . . . 44
- 9.3 Randomized Prim . . . 46
- 9.4 Recursive Backtracking . . . 47
- 9.5 Eller . . . 48

List of Figures

1 Elements . . . 4
2 Maze Example . . . 6
3 Examples of maze generated using Recursive Backtracking. . . . 9
4 Aldous-Broder Maze Example . . . 10
5 Wilson Maze Example . . . 11
6 Randomized Prim Maze Example . . . 13
7 Recursive Backtracking Maze Example . . . 15
8 Eller Maze Example . . . 17
12 Toy Example empirical Time complexity . . . 21
13 Empirical Time complexity . . . 28
14 Inherent Metrics Analysis . . . 32
15 20×20 solved Aldous-Broder maze. . . . 34
16 20×20 solved Wilson maze. . . . 34
17 20×20 solved Randomized Prim maze. . . . 35
18 20×20 solved Recursive Backtracking maze. . . . 35
19 20×20 solved Eller maze. . . . 36
20 Agent Metrics Analysis . . . 37
21 Aldous-Broder Tree . . . 40
22 Wilson Tree . . . 41
23 Prim Tree . . . 41
24 Recursive Backtracking Tree . . . 42
25 Eller Tree . . . 42

1 Introduction

Mazes have been part of human history for millennia. They appear in ancient pyramids to conceal treasures, in architectural designs for various purposes such as storing goods or disorienting enemies, and even with symbolic or spiritual significance. Mazes have also been closely associated with art. They have been depicted in paintings, embedded in floor designs, or constructed in gardens as decorative elements. However, their application in scientific contexts emerged only much later. The use of mazes in a "modern" sense is relatively recent; for example, it was only in the 1890s that they began to be used in animal psychology experiments.

Today, mazes are employed across a wide range of disciplines, from biology to psychology, mathematics to robotics, game design to artificial intelligence, and even as simple logic puzzles for enthusiasts.

Another notable, though more abstract, application of mazes is in data storage. As human knowledge and technological advancements continue to expand, the volume of data that must be processed and stored is increasing at an exponential rate. Efficiently locating relevant and valuable information among billions of data points has become an essential challenge. To address this, the development of fast and efficient maze-solving algorithms is increasingly important. Moreover, the manner in which data are structured and stored can significantly influence retrieval speed and efficiency, highlighting the role of maze generation algorithms in this context.

1.1 What is a maze?

In the context of our project, a maze is defined as a logical puzzle or a virtual environment for video games. It is composed of cells (positions where a player may pause before choosing the next move) and links, which may be closed (walls: Red lines in Figure 1) or open (passages). Based on the number of walls surrounding a cell, we distinguish three categories:

- **Dead End**: a cell with three walls (Blue square in Figure 1);
- **Corridor**: a cell with two walls (Red lines in Figure 1);
- **Junction (or Intersection)**: a cell with fewer than two walls (Blue square in Figure 1);

In this study, we focus on the generation of perfect mazes; therefore, cells with four walls (i.e., isolated cells) are not produced.

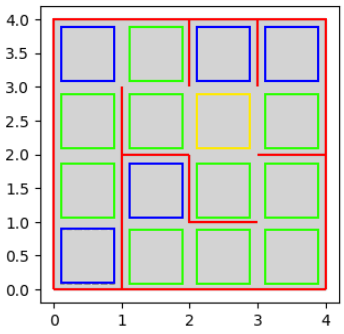


Figure 1: A 4x4 maze generated with Randomized Prim algorithm.

1.2 Objectives of the Study

In this paper, we focus on a specific application: maze generation for games. We examine maze generation algorithms from the perspective of gameplay, investigating how effectively each algorithm can create maps that are both enjoyable and challenging.

The following five widely recognized algorithms were selected for our study:

- Aldous–Broder algorithm
- Wilson's algorithm
- Randomized Prim's algorithm
- Recursive Backtracking algorithm
- Eller's algorithm

Each algorithm follows a unique approach to path construction, resulting in notable differences in generation time and maze structure. The objective of this paper is to evaluate the performance of these algorithms in terms of playability and to determine which produces the most engaging and challenging mazes for players.

After presenting each algorithm and the characteristics of its generated mazes, we demonstrate practical applications, such as maze solving using the A* algorithm. We then provide a comparative analysis of each algorithm's strengths and weaknesses, again employing the A* algorithm as the solving agent.

2 Mathematical Modeling

Before jumping into maze generation algorithms, understanding the mathematics behind them is crucial, and can help to understand why the generated mazes have such as differences between them.

2.1 Graph Theory

Graph theory is the core of generation and solution of mazes. In mathematics, graph theory is the study of graphs, theoretical structures used to model relations between objects.

In the maze generation context, we use graphs to model the underlying grid structure:

- **Vertices (V)**: represent cells (rooms/spaces) in the maze
- **Edges (E)**: represent potential passages between adjacent cells
- **Maze**: created by selecting a subset of these potential passages

We now need to introduce some definitions to clarify what this project is going to cover:

1. **Def (Undirected Graph)**: An undirected graph is an ordered pair G = (V,E) consisting of:

   - A non-empty set V (called the vertices)
   - A set E (called the edges) of two-element subsets of V, where each edge {u,v} connects vertices u and v

Example: Consider a graph with 4 vertices {a, b, c, d} and five edges {{a,b}, {a,c}, {b,c}, {b,d}, {c,d}}

2. **Def (Path)**: A **path** in a graph is a finite sequence of vertices:

$$P = (v_0, v_1, \ldots, v_k)$$

such that for each $i = 0, 1, \ldots, k-1$, the edge $\{v_i, v_{i+1}\} \in E$.

- A *simple path* is a path where all vertices (and therefore edges) are distinct.
- A *cycle* is a path where $v_0 = v_k$, and all other vertices are distinct.

3. **Def (Connected Graph)**: A graph G is said to be **connected** if for every pair of vertices $u, v \in V$, there exists a path from u to v. That is,

$$\forall u, v \in V, \exists \text{ a path } (u = v_0, v_1, \ldots, v_k = v) \text{ with } \{v_i, v_{i+1}\} \in E.$$

4. **Def (Simple Graph)**: A graph is called **simple** if it has no loops (edges from a vertex to itself) and no multiple edges between the same pair of vertices.

5. **Def (Degree)**: The degree of a vertex v, denoted deg(v), is the number of edges incident to v. In our paper, vertices have minimum degree of 1 (for dead ends), and maximum degree of 4 (for intersections or "junctions").

6. **Def (Tree)**: A **tree** is a connected graph with no cycles. Equivalently, a tree with n vertices has exactly n-1 edges.

7. **Def (Spanning Tree)**: Let $G = (V, E)$ be a finite, connected, undirected graph. A **spanning tree** of G is a subgraph $T = (V, E_T)$ such that:

- T is connected and acyclic (i.e., a tree)
- $E_T \subseteq E$ and $|E_T| = |V| - 1$
- T includes all vertices of G

8. **Def (Perfect Maze)**: A **perfect maze** is a maze with the following properties:

- There is exactly one path between any two points in the maze
- The maze is connected (you can reach any point from any other point)
- There are no cycles (no loops in the maze)

Mathematically, a perfect maze corresponds to a spanning tree of the underlying grid graph.

9. **Def (Uniform Spanning Tree)**: A **uniform spanning tree (UST)** is a spanning tree chosen uniformly at random from the set $\mathcal{T}(G)$ of all spanning trees of G. That is, each spanning tree $T \in \mathcal{T}(G)$ is selected with probability:

$$\mathbb{P}(T) = \frac{1}{|\mathcal{T}(G)|}.$$

where $|\mathcal{T}(G)|$ is the total number of spanning trees.

In other words, each possible perfect maze has an equal probability of being generated, resulting in mazes with no inherent bias toward particular structures or patterns.

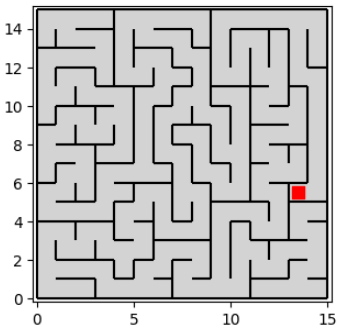


Figure 2: A 15x15 perfect maze, generated with Recursive Backtracking algorithm. The red square is the starting generation point.

In this project, we focus on generating perfect mazes, which means our algorithms create spanning trees of grid graphs. The resulting structures are:

- **Undirected**: passages can be traversed in both directions
- **Connected**: every cell is reachable from every other cell
- **Acyclic**: no loops exist in the maze

2.2 Random Walks

Some of the algorithms presented in this project need additional mathematical concepts to be understood properly. While some algorithms generate mazes through deterministic processes, others like Aldous-Broder and Wilson use random walks to generate uniform spanning trees.

The mathematical notion that enables the generation of unbiased mazes is the **random walk**. In mathematics, a random walk is a stochastic process that describes a path consisting of a succession of random steps on some mathematical space.

For random walk analysis, we need to work with matrix representations of graphs:

Let $G = (V, E)$ be a finite, simple, undirected graph with vertex set $V = \{v_1, \ldots, v_n\}$ and edge set E. Let $A \in \mathbb{R}^{n \times n}$ denote the adjacency matrix of G, where:

$$A_{ij} = \begin{cases} 1 & \text{if } \{v_i, v_j\} \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Define the degree matrix D as a diagonal matrix where $D_{ii} = \deg(v_i)$, the degree of vertex v_i. Since we work with connected graphs, all vertices have positive degree, so D^{-1} exists.

Def (Random Walk): A **random walk** on the graph G is a discrete-time stochastic process $\{X_t\}_{t \geq 0}$, where $X_t \in V$ is the location of the walk at time t. At each step, the walker moves from its current vertex to a randomly chosen neighbor according to the transition probability matrix.

The transition probability matrix $P \in \mathbb{R}^{n \times n}$ is defined as:

$$P = D^{-1}A.$$

This gives the transition probabilities:

$$P_{ij} = \mathbb{P}(X_{t+1} = v_j \mid X_t = v_i) = \begin{cases} \frac{1}{\deg(v_i)} & \text{if } \{v_i, v_j\} \in E, \\ 0 & \text{otherwise.} \end{cases}$$

Application to Maze Generation:

In maze generation, the underlying grid graph represents all possible connections in a rectangular lattice. The vertices of this graph can be categorized as follows:

- **Interior vertices**: Vertices not on the boundary of the grid. Each has degree 4, with edges connecting to the up, down, left, and right neighbors.
- **Boundary vertices**: Vertices located on the edges or corners of the grid. Their degrees vary:
  - Edge vertices have degree 3.
  - Corner vertices have degree 2.

During maze generation, edges are removed from this full grid graph to form passages and dead ends. In the case of a perfect maze, the resulting structure is a spanning tree of the original grid graph, where vertex degrees typically become 1 (for dead ends), 2 (for straight passages), or occasionally 3 or 4 (for junctions).

The random walk operates on this grid graph structure, using uniform transition probabilities to move from one vertex to an adjacent neighbor. A key insight is that certain random walk-based algorithms, such as the Aldous-Broder and Wilson algorithms, naturally generate uniform spanning trees when they terminate.

In other words, these processes ensure that every possible perfect maze (i.e., every spanning tree of the underlying graph) has equal probability of being generated. This eliminates any structural bias that might arise from deterministic maze generation methods, producing mazes that are both perfect and uniformly distributed over the space of all possible solutions.

2.3 Metrics

In this section, we present the different metrics used to analyze the generated mazes. While the difficulty of a maze can often be intuitively estimated by visually inspecting it, such subjective judgment is not sufficient for rigorous analysis. Therefore, we categorize the metrics into two distinct groups: *Maze Inherent Metrics* and *Solver-Based Metrics*.

Although this paper primarily focuses on maze generation, agent-based solving strategies can provide useful information for certain metrics. Below, we define and justify the metrics considered in our analysis.

2.3.1 Maze Inherent Metrics

These metrics describe the structural and topological properties of the maze as generated, independent of any solving strategy:

- **Generation Time:** While Section 5.1 discusses the theoretical time complexity of each algorithm, this metric captures the actual time required to generate the maze in practice. It highlights empirical performance differences among implementations, which may be affected by language, data structure choices, or hardware.
- **Dead Ends:** This metric counts the number of cells with only one adjacent open passage (i.e., three surrounding walls). Dead ends are relevant to maze difficulty, as they increase the likelihood of backtracking during traversal. A higher number of dead ends generally correlates with a higher solving complexity.
- **Junctions (Intersections):** This represents the number of cells with three or more open passages (i.e., one or no walls). Such cells act as decision points during solving. A greater number of junctions increases the chances of making incorrect choices, thereby raising the perceived complexity of the maze.
- **Total Nodes**: it is the sum of the relevant nodes in the maze, which are dead ends and junctions:

$$\text{Total Nodes} = |\text{Dead Ends}| + |\text{Junctions}|$$

- **Efficiency**: this metrics represents the generation of nodes over the time. Highest the value, highest the generation of decision nodes.

$$\text{Efficiency} = \frac{|\text{Dead Ends}| + |\text{Junctions}|}{\text{Generation Time}}$$

- **Complexity Ratio**: this metric provides insight into the maze's structure: a higher value indicates more false paths and potential backtracking, while a lower value suggests a more interconnected and navigable maze.

$$\text{Complexity Ratio} = \frac{|\text{Dead Ends}|}{|\text{Junctions}|}$$

2.3.2 Solver-Based Metrics

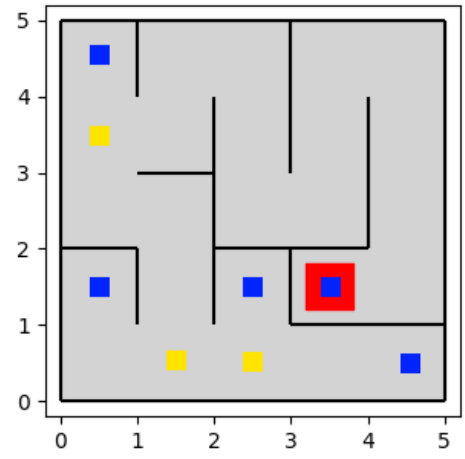
While inherent metrics help us understand the structural complexity of a maze, solver-based metrics provide an objective measure of the actual difficulty experienced by an agent navigating the maze. Although different solving algorithms may yield different results, in this work we adopt the A* algorithm to retrieve performance metrics and solve the generated mazes.

- **Steps:** The total number of steps taken by the agent to find a valid path from the start to the goal. While this metric often correlates with difficulty, more steps typically imply a more complex maze, it may be skewed by the maze's structure. As discussed in Section 3, some algorithms introduce structural biases that may force longer paths even when the solution is straightforward.
- **Visited Cell Ratio:** The percentage of all maze cells that were visited during the search process. A higher value generally indicates less efficient exploration, implying greater difficulty in locating the correct path.
- **Solution Time:** The elapsed time required to compute the path from start to goal. Like the number of steps, higher time may indicate increased difficulty, though it may also reflect implementation details or maze-specific biases.
- **Visited Nodes**: sum of visited junctions and dead ends.
- **Nodes Coverage**: gives a normalized score representing the proportion of key decision points explored during maze solving.

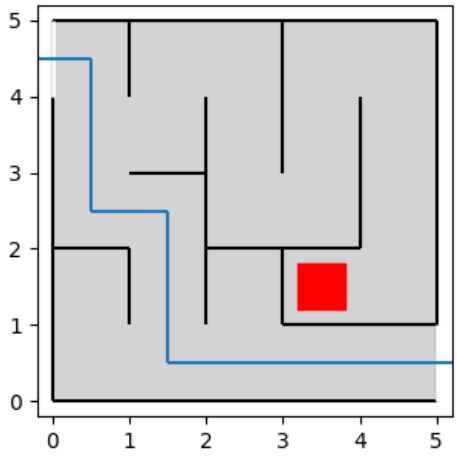
$$\text{Node Coverage} = \frac{|\text{Visited Junctions}| + |\text{Visited Dead Ends}|}{|\text{Total Nodes}|}$$

- **Corridors Visited (%)**: This metric shows the percentage of visited cells by the agent that are corridors.
- **Steps/s**: The average number of steps taken by the agent per second.
- **Steps/Visited Node**: The average number of steps required to discover a new decision node:

$$\text{Steps/Visited Node} = \frac{|\text{Steps}|}{|\text{Visited Nodes}|}$$



(a) 5x5 maze. Blue: Dead Ends. Yellow: Intersections



(b) 5x5 maze. Blue line is the solution path. Red square is origin of generation

Figure 3: Examples of maze generated using Recursive Backtracking.

3 Algorithms Presentation

In this section, we will present the four algorithms, starting from the reason why they were created and/or used. After that, pseudo-code of the main function will be reported. Finally, each algorithm subsection will end with some maze, generated with the presenting algorithm.

3.1 Aldous-Broder

The **Aldous-Broder algorithm** is one of the simplest methods for generating a perfect maze, yet it is also among the most inefficient. Its inefficiency stems from its random walk traversal strategy: at each step, the algorithm selects a neighbor of the current cell uniformly at random, regardless of whether that neighbor has already been visited. As a result, the walk may revisit many cells multiple times before covering the entire graph, leading to potentially very long generation times, especially for large grids.

Despite this drawback, Aldous-Broder guarantees the generation of a uniform spanning tree, just like Wilson's algorithm.

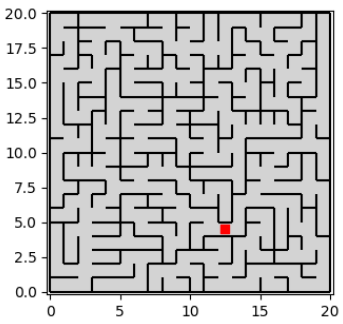


Figure 4: A 20x20 maze generated with Aldous-Broder algorithm.

Algorithm 1 Aldous–Broder Algorithm for Uniform Spanning Tree

Require: Grid dimensions (rows and columns)
  Choose a random cell from the grid (call it *Current*)
  Mark Current as Visited
  while Visited $\neq$ Grid **do**
    Find neighbours of Current
    Choose randomly a neighbour
    if neighbour is not visited **then**
      Mark neighbour as visited
      remove wall between neighbour and current
    end if
  end while
  Return completed maze

Properties:

Aldous-Broder is conceptually simple and easy to implement, which can be a significant advantage.

However, its major drawback lies in its slow generation time and inefficiency, especially for large grids. Due to its reliance on an unrestricted random walk, it often revisits already explored cells multiple times before completing the maze. This results in a long and computationally expensive process, making Aldous-Broder unsuitable for performance-critical applications.

Thus, its practical use must be carefully considered in light of its performance limitations.

Python Code – Main Function.
In this section, we present the Python implementation of Aldous-Broder's `main()` function. All auxiliary functions required for execution are provided in Appendix 9.

```python
def main(self):
        """Main function. It generates the maze using all other functions"""

        # Generated all necessary lists and dictionaries
        self.grid, self.maze_map, self.neighbour_caches =
            self.create_grid(self.step_by_step, self.visualizer)

        self.start = random.choice(list(self.grid))
        previous_position = self.start # leaves open the passage of the step before

        total_cells = len(self.grid)
        while len(self.visited_cells) < total_cells:

            next_position = self.next_cell(previous_position)
            if next_position not in self.visited_cells:
                self.visited_cells.add(next_position) # if next cell is not visited,
                    it marks the cell as visited
                self.place_wall(next_position, previous_position)

                if self.step_by_step:
                    self.visualizer.represent_cell(self.rows, next_position,
                        'lightgrey') # show the cell in the plot

            else:
                if self.step_by_step:
                    self.visualizer.represent_cell(self.rows, next_position, color =
                        'red') # show the step
                    self.visualizer.show_step(self.plot_size)
                    self.visualizer.represent_cell(self.rows, next_position,
                        'lightgrey') # shows the visited cell

            previous_position = next_position
        if self.step_by_step:
            self.visualizer.visualize_starting_point(self.rows, self.start, 'red')
        return
```

Listing 1: Main function of Aldous-Broder Maze generation algorithm

3.2 Wilson

Wilson's algorithm generates mazes by using loop-erased random walks, which results in unbiased and uniformly distributed spanning trees. This method improves upon the inefficiencies of algorithms like Aldous-Broder by avoiding redundant revisits to already-visited nodes through the loop-erasure process.

As with Aldous-Broder, the resulting maze is guaranteed to be a uniform spanning tree.

For a deeper understanding of random walks and uniform spanning trees, refer to the Mathematical Model Formulation section 2.2).

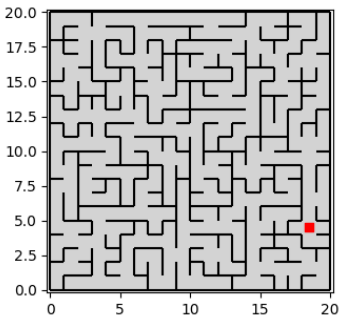


Figure 5: A 20x20 maze generated with Wilson algorithm.

Algorithm 2 Wilson's Maze Generation Algorithm

Require: Grid dimensions (rows and columns)
Choose a random cell from the grid and mark it as *End*
Mark *End* as visited
while there are unvisited cells **do**
    Choose a random unvisited cell (call it *Current*)
    Initialize an empty list called *Path*
    Append *Current* to *Path*
    while *Current* is not visited **do**
        Find all unvisited neighbours of *Current*
        Choose a random neighbour (call it *Next*)
        if *Next* is already in *Path* **then**
            Erase the loop by removing all steps after the first occurrence of *Next*
        else
            Append *Next* to *Path*
        end if
    end while
    for each pair of consecutive cells in *Path* **do**
        Remove the wall between them
        Mark the cells as visited
    end for
end while
return the completed maze

Properties:

As mentioned at the beginning of this section, Wilson's algorithm generates unbiased mazes corresponding to uniform spanning trees. This makes it particularly valuable in applications requiring fairness and statistical consistency.

However, one major drawback is that the algorithm can be very slow, especially for large mazes. This inefficiency stems from the use of loop-erased random walks: while they ensure unbiasedness, they often require long paths to be generated and then shortened by removing loops, which can be computationally expensive.

Below are some examples of mazes generated using Wilson's algorithm:

Python Code – Main Function.
In this section, we present the Python implementation of Wilson's `main()` function.

```
def main(self):
        """Main function. It generate the maze using all other functions"""

        # Generated all necessary lists and dictionaries
        self.grid, self.maze_map, self.neighbour_caches =
            self.create_grid(self.step_by_step, self.visualizer)

        free_positions = self.grid.copy()
        end, free_positions = self.start_position(free_positions) # extract origin
            and remove it from the next possible choices
        self.visited_cells.add(end)

        # define end and include it in the final path
        if self.step_by_step:
```

```
13              self.visualizer.represent_cell(self.rows, end, 'lightgrey')
14
15          self.start, free_positions = self.start_position(free_positions) # first cell
                from which start mining
16          current_path = self.path_miner(self.start) # return the new path that have to
                be inclused in the visited list
17          self.place_wall(current_path) # place walls
18
19          free_positions =self.free_cells(free_positions, current_path) # retrieve
                remaining free cells
20
21          total_cells = len(self.grid)
22          while len(self.visited_cells) != total_cells:
23
24              next_cell, free_positions = self.start_position(free_positions) # define
                    starting position of new mining path
25              current_path = self.path_miner(next_cell) # finds new mining path
26              self.place_wall(current_path) # places walls
27              free_positions = self.free_cells(free_positions, current_path) # finds
                    not visited cells
28
29          if self.step_by_step:
30              self.visualizer.visualize_starting_point(self.rows, self.start, 'red') #
                    visualize origin of the maze
31          return
```

Listing 2: Main function of Randomized Wilson Maze generation algorithm

3.3 Randomized Prim's Algorithm

The **Randomized Prim algorithm** is a randomized variant of **Prim's algorithm**, which was originally developed in 1957. However, it was also independently proposed earlier by Edsger W. Dijkstra in 1956 in a slightly different form. The algorithm is classically used to find a minimum spanning tree (MST) in a weighted, undirected graph, that is, a subset of edges that connects all vertices with the minimal total edge weight.

In its original form, Prim's algorithm starts from an arbitrary vertex and repeatedly adds the shortest edge connecting the growing tree to a new vertex. This greedy approach ensures an optimal solution for graphs with real-valued edge weights.

In the context of maze generation, the algorithm is adapted into a randomized version where:

- The initial starting vertex is chosen at random.
- All edges (i.e., connections between a cell and its neighbors) are considered to have equal weight. When multiple candidate edges exist, one is selected randomly.

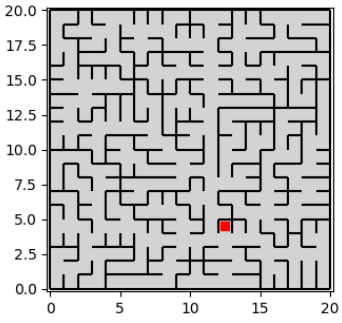


Figure 6: A 20x20 maze generated with Randomized Prim algorithm.

This randomization eliminates bias in path selection and results in a perfect maze structure, although not necessarily a uniformly random one — unlike Wilson's or Aldous-Broder algorithms.

Algorithm 3 Prim's Maze Generation Algorithm

Require: Grid dimensions (rows and columns)
Choose a random cell from the grid (call it *Start*)
Mark *Start* as visited
Add all unvisited neighbors of *Start* to a set called *Frontier*
while *Frontier* is not empty **do**
    Choose a random cell from *Frontier* (call it *Current*)
    Let *VisitedNeighbors* be the visited neighbors of *Current*
    Choose one *VisitedNeighbor* at random
    Remove the wall between *Current* and the chosen *VisitedNeighbor*
    Mark *Current* as visited
    Add all unvisited neighbors of *Current* to *Frontier*
    Remove *Current* from *Frontier*
end while
Return the completed maze

Properties:

Mazes generated by the Randomized Prim algorithm are characterized by an abundance of dead ends and short cul-de-sacs. This property arises naturally from the way the algorithm expands the maze: by randomly selecting frontier edges, it tends to form many small branches that terminate quickly.

In the context of large mazes, this high density of short dead ends can make the maze appear more complex and challenging at a glance. This visual complexity can be particularly advantageous in video game design, where increased branching and false paths may contribute to a higher perceived difficulty or a more engaging player experience.

An example of a maze generated using Prim's algorithm is shown below.

Python Code – Main Function.

In this section, we present the Python implementation of Randomized Prim's `main()` function. All auxiliary functions required for execution are provided in Appendix 9.

```
def main(self):
        """Main function. It generates the maze using all other functions"""

        frontier = set()
        self.grid, self.maze_map, self.neighbour_caches =
            self.create_grid(self.step_by_step, self.visualizer)
        free_positions = self.grid.copy() # set the initial non visited positions

        self.start, free_positions = self.start_position(free_positions, color =
            'lightgrey') # extract origin and remove it from the next possible choices
        self.visited_cells.add(self.start)

        if self.step_by_step:
            self.visualizer.show_step(self.plot_size)

        neighbours = self.neighbourhood(self.start, free_positions) # find neighbours
            of the origin
        frontier = self.check_frontier(frontier, neighbours) # updates the frontier

        while len(frontier) != 0: # until the frontier is empty

            next_position, frontier = self.start_position(frontier, color =
                'lightgrey') # new step
```

```
            self.visited_cells.add(next_position)
            visited_neighbours = self.neighbourhood(next_position, self.visited_cells)

            passage = random.choice(visited_neighbours) # randomly chooses a cell
                from which maintain open the passage
            self.place_wall(next_position, passage) # places walls

            neighbours = self.neighbourhood(next_position, self.grid) # find
                neighbours of the current cell
            frontier = self.check_frontier(frontier, neighbours) # updates the
                frontier

        if self.step_by_step:
            self.visualizer.visualize_starting_point(self.rows, self.start, 'red')
        return
```

Listing 3: Main function of Randomized Prim Maze generation algorithm

3.4 Recursive Backtracking (Depth-First Search)

One of the simplest and oldest algorithms for maze generation is the **Recursive Backtracker**, which belongs to the broader class of Depth-First Search (DFS) algorithms.

Depth-First Search is a fundamental algorithm used for traversing or searching tree or graph data structures. It begins at a chosen root node (or an arbitrary starting node in the case of a graph) and explores as far as possible along each branch before backtracking. To manage this traversal, DFS typically uses a stack (explicitly or implicitly via recursion) to keep track of the nodes visited along the current path, enabling the algorithm to backtrack when it reaches a dead end.

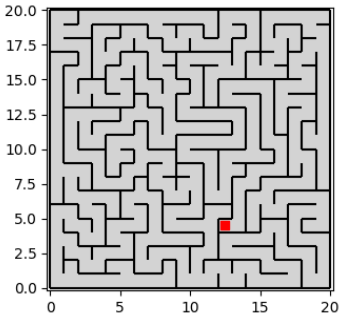


Figure 7: A 20x20 maze generated with Recursive Backtracking algorithm.

Algorithm 4 Recursive Backtracking Maze Generation Algorithm

Require: Row and Cols dimension
  Choose a random starting cell and mark it as visited
  Initialize an empty stack
  Push the starting cell onto the stack
  while stack is not empty **do**
    Let *current* be the top cell on the stack
    Find all unvisited neighbors of *current*
    if there are unvisited neighbors **then**
      Choose one randomly (call it *next*)
      Remove the wall between *current* and *next*
      Mark *next* as visited
      Push *next* onto the stack
    else
      Pop the top cell from the stack
    end if
  end while
  Return Maze

Properties:

The Recursive Backtracking (DFS) algorithm is one of the simplest and most straightforward methods for generating mazes.

Mazes produced by this algorithm typically have a low branching factor and are characterized by the presence of many long, winding corridors. This structure arises naturally from the algorithm's behavior: it explores as far as possible along each path before backtracking, leading to deep, snake-like passages with relatively few branches.

As a result, these mazes often feel linear and may be easier to solve visually due to their predictable structure — although they can appear complex due to their length and depth.

Python Code – Main Function.

In this section, we present the Python implementation of Recursive Backtracking's `main()` function. All auxiliary functions required for execution are provided in Appendix 9.

```
def main(self):
        """Main function. It generates the maze using all other functions"""

        self.grid, self.maze_map, self.neighbour_caches =
            self.create_grid(self.step_by_step, self.visualizer)
        free_positions = self.grid.copy()

        self.start, free_cells = self.start_position(free_positions, color = 'red') #
            Choose starting positin

        stack = [self.start]  # Stack for backtracking
        self.visited_cells.add(self.start)

        while len(stack) != 0:
            current_cell = stack[-1]  # Peek at top of stack
            neighbours = self.neighbourhood(current_cell, free_positions) # get
                unvisited neighbors
            unvisited_neighbours = []

            for neigh in neighbours:
                if neigh not in self.visited_cells:
                    unvisited_neighbours.append(neigh)

            if len(unvisited_neighbours) != 0:
                next_cell = self.next_cell(current_cell, unvisited_neighbours) #
                    choose random unvisited neighbor
                self.place_wall(next_cell, current_cell)
                stack.append(next_cell) # add to stack and mark as visited
                self.visited_cells.add(next_cell) # adds the cell to visited cells
                free_positions.remove(next_cell) # remove next cell from the free
                    positions

            else: # if there are no free neighbours available
                backtrack_cell = stack.pop() # Dead end - backtrack

                if self.step_by_step: # Visualize backtracking
                    self.visualizer.represent_cell(self.rows, backtrack_cell, 'red')
                    self.visualizer.show_step(self.plot_size)
                    self.visualizer.represent_cell(self.rows, backtrack_cell,
                        'lightgrey') # Change back to show it's part of the maze

        if self.step_by_step:
            self.visualizer.visualize_starting_point(self.rows, self.start, 'red')  #
                Start cell in green
        return
```

Listing 4: Main function of Recursive Backtracking Maze generation algorithm

3.5 Eller's Algorithm

Eller's algorithm is a highly efficient and procedural maze generation technique that constructs the maze row by row, ensuring connectivity while avoiding cycles through the use of a disjoint-set data structure.

At each step, the algorithm maintains information about which cells in the current row are connected, allowing it to merge sets appropriately when carving passages vertically to the next row. This enables the algorithm to generate mazes of arbitrary height with only constant memory usage per row, making it especially well-suited for online maze generation, where the full grid does not need to be stored in memory at once.

This property makes Eller's algorithm ideal for applications with limited memory resources or for generating mazes in streaming environments.

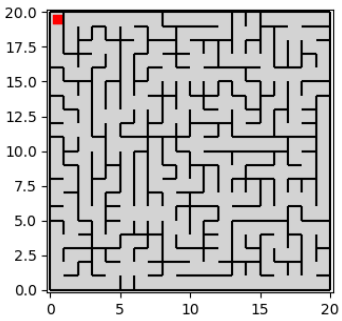


Figure 8: A 20x20 maze generated with Eller algorithm.

Algorithm 5 Eller's Maze Generation Algorithm

Require: Grid dimensions (rows and columns)
Start from the top-left cell (call it *Start*)
for each row in the grid **except the last one do**
    Assign a unique set ID to each cell in the row that doesn't already have one
    for each pair of adjacent cells in the row **do**
        if the cells belong to different sets **and** random chance allows **then**
            Merge the sets: assign the same set ID
            Remove the wall between the two cells
        end if
    end for
    for each unique set in the row **do**
        Randomly select one or more cells from the set
        for each selected cell **do**
            Remove the wall between the current cell and the cell below
            Assign the cell below to the same set
        end for
    end for
end for
Handle the final row:
for each pair of adjacent cells **do**
    if they belong to different sets **then**
        Merge the sets and remove the wall
    end if
end for

Properties: Eller's algorithm introduces a slight horizontal bias due to its row-by-row processing. Since each row is handled sequentially and horizontal connections

are naturally favored, vertical connections must be explicitly created. This results in mazes that often exhibit a stronger horizontal flow. Additionally, this gives the maze a visual pattern that can influence solving difficulty depending on the solver's strategy. Finally, thanks to its linear time complexity (discussed in the Time Complexity Analysis section 5.1), Eller's algorithm is highly efficient and well-suited for fast maze generation, even for large grids. This efficiency, combined with its low memory footprint, makes it particularly useful for real-time or memory-constrained applications.

Python Code – Main Function.
In this section, we present the Python implementation of Eller's `main()` function. All auxiliary functions required for execution are provided in Appendix 9.

```
def main(self):
        """Main function. It generates the maze using all other functions"""

        self.grid, self.maze_map, self.neighbour_caches =
            self.create_grid(self.step_by_step, self.visualizer)
        self.start = (0, 0)

        for row in range(self.rows):
            if row == 0:
                self.row_sets = self.assign_sets(row) # first set_id assignement

            self.merge_sets(row)

            if row < self.rows - 1: # until last row
                self._create_vertical_connections(row)

            else:
                self.final_merge()

        if self.step_by_step:
            self.visualizer.visualize_starting_point(self.rows, self.start, 'red')  #
                Start cell in green
        return
```

Listing 5: Main function of Eller Maze generation algorithm

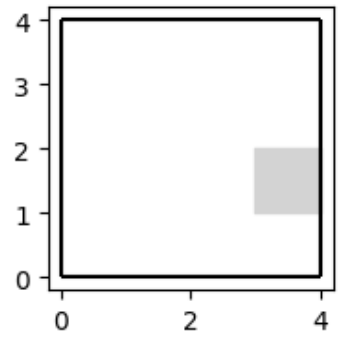
4 A simple Maze Generation

4.1 Step by step generation

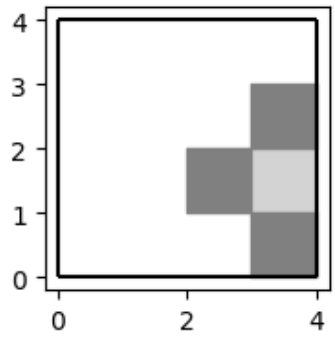
To illustrate how maze generation algorithms work in practice, we now walk through an example using the first algorithm we implemented: **Randomized Prim's algorithm**.

At each step, the algorithm randomly selects a cell from the **frontier set**, the set of unvisited cells adjacent to already visited ones, with all elements having equal probability of being chosen. This random selection ensures that the resulting maze is unbiased, while still maintaining the efficiency and simplicity of the original approach.(see 3.3).

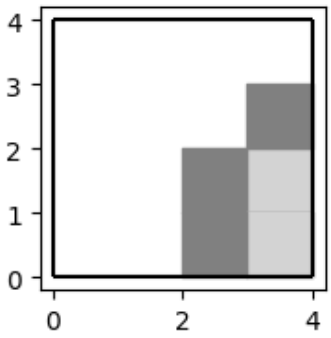
Here is how it works graphically:



Start from a randomly chosen point.

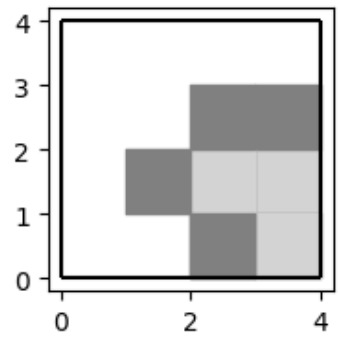


Insert in frontier (grey) all neighbours not visited yet.

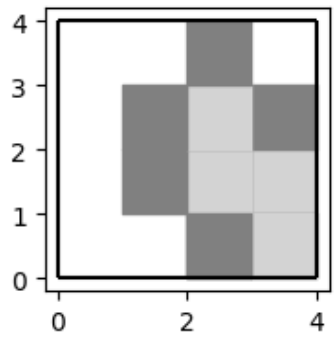


Chooses the new step within the frontier set.

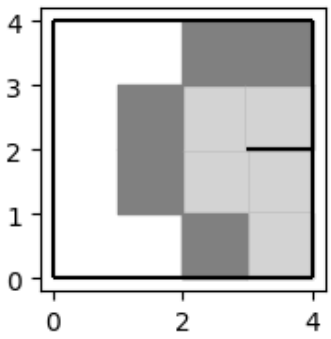
Continuing with the above steps:



Choose a new frontier point, and mark it as visited.

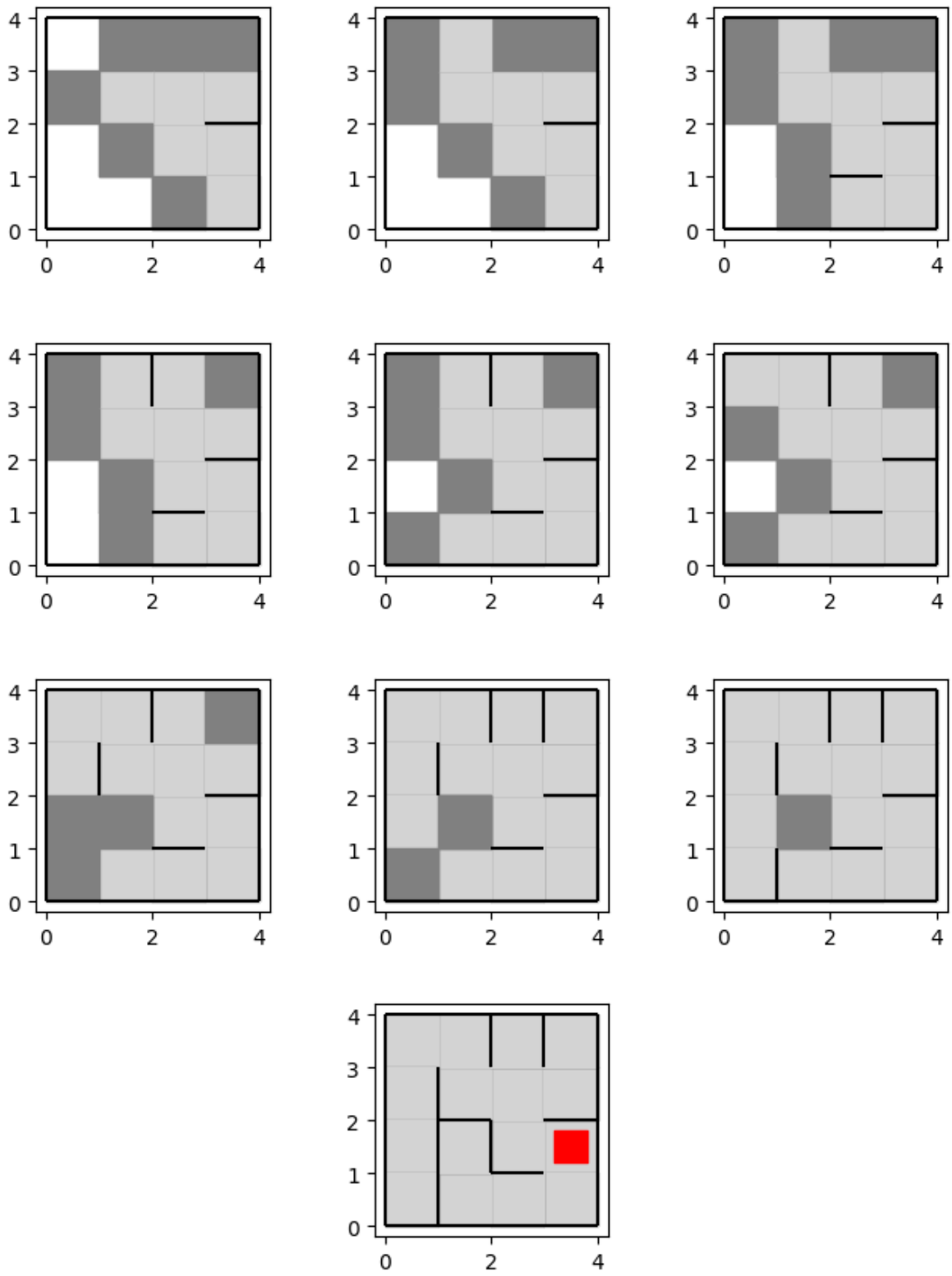


Again choose a new frontier point, and mark it as visited.



When an adjacent cell was already visited, place a wall.

Let's just keep it going to the end:



Conclusion of Maze generation and Origin of generation(red square)

Generation Time Analysis

To analyze the time performance of maze generation, we report the time required to generate mazes with dimensions ranging from 2×2 up to 100×100, using the Randomized Prim algorithm.

It is important to note that in this preliminary test, only a single maze is generated for each grid size. Therefore, while this provides a general sense of the algorithm's behavior, the results may not be statistically robust.

A more comprehensive analysis, based on multiple maze generations per dimension and including average, minimum, and maximum generation times, will be presented in the full *Performance Analysis* section.(Section 6).

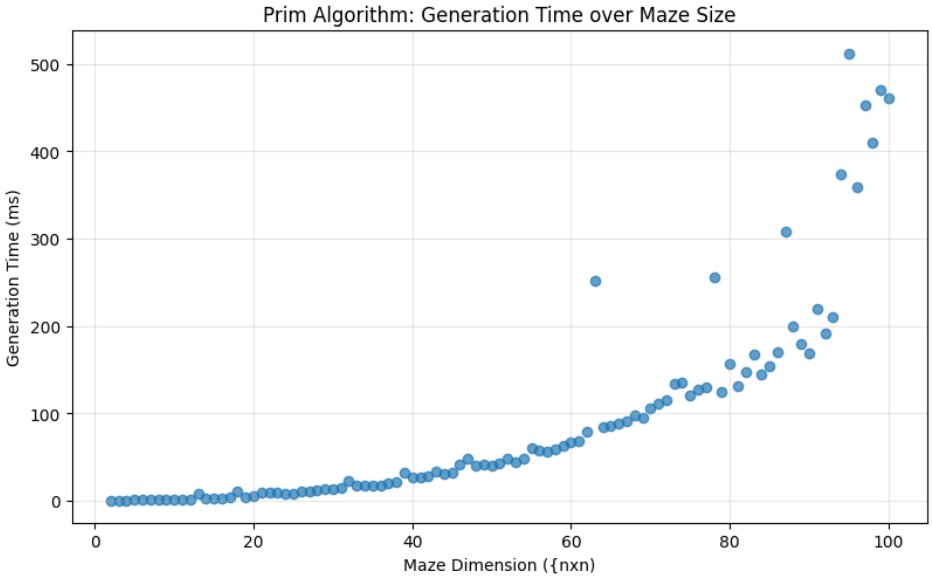


Figure 12: Generation time for maze sizes from 2×2 to 100×100 using Randomized Prim (single iteration per size).

5 Time and Space Complexity Analysis

In this section, we analyze the time and space complexities of the maze generation algorithms. We now focus on the analysis of pseudocode. This abstraction allows for a clearer understanding of each algorithm's theoretical efficiency and general behavior, independent of specific implementation details.

5.1 Time Complexity Analysis

To better readability and to avoid redundancies, below common statements are listed with their time complexities:

1. Choosing a starting cell randomly from a list has, on average, time complexity $\mathcal{O}(1)$, since Python lists are implemented as arrays and selecting an element by index is a constant-time operation.

$$\textit{Choose a random element from a list} \longrightarrow \mathcal{O}(1)$$

2. Different is the case in which an element have to be chosen from a set. Indeed firstly the set have to be converted in a list, which has cost $\mathcal{O}(n)$.

$$\textit{Choose a random element in a set} \longrightarrow \mathcal{O}(n)$$

3. Adding an element to a stack also takes $\mathcal{O}(1))$ time, on average. Stacks in Python are typically implemented using lists, and the `append()` method adds elements to the end of the array, which is a constant-time operation due to direct memory addressing.

$$\textit{Add an element onto the stack} \longrightarrow \mathcal{O}(1)$$

4. Finding all unvisited neighbors of a cell has time complexity $\mathcal{O}(1)$, as each cell has at most four neighbors (up, down, left, right), which is a fixed and small number.

$$\textit{Find neighbors of a cell} \longrightarrow \mathcal{O}(1)$$

5. Checking whether an element has already been visited, using a `set`, also takes $\mathcal{O}(1)$ time per check.

$$\textit{Check if an element is in a set} \longrightarrow \mathcal{O}(1)$$

6. In case of checking whether an element in a `list`, also takes $\mathcal{O}(n)$, since have to be checked every element in the list.

$$\textit{Check if an element is in a list} \longrightarrow \mathcal{O}(1)$$

7. Removing the wall between two adjacent cells is again a constant-time operation, $\mathcal{O}(1)$, since it typically involves updating a fixed attribute or flag in the data structure representing the cell.

$$\textit{Remove the wall between current and next} \longrightarrow \mathcal{O}(1)$$

5.1.1 Aldous-Broder

Assumptions:

- Visited and Unvisited cells are `sets`.

```
1: Choose a random cell from the grid (call it Current) ▷ O(n): Convert set to list + random choice
2: Mark Current as Visited                                                          ▷ O(1)
3: while Visited ≠ Grid do
4:     Find neighbours of Current                                  ▷ O(1): max 4 neighbours
5:     Choose randomly a neighbour                                                  ▷ O(1)
6:     if neighbour is not visited then                            ▷ O(1): check element in a set
7:         Mark neighbour as visited                               ▷ O(1): add element in a set
8:         remove wall between neighbour and current                                ▷ O(1)
9:     end if
10: end while
```

While the time complexity of a single iteration inside the loop is $\mathcal{O}(1)$, the overall complexity is not simply $n \times \mathcal{O}(1) = \mathcal{O}(n)$, because a cell can be visited multiple times before all cells are marked as visited. This is due to the nature of the random walk used in Aldous-Broder's algorithm.

The total number of steps required to visit every cell at least once (known as the *cover time*) is not linear in n, but rather depends on the structure of the grid. For a 2D grid, the expected number of steps is $\mathcal{O}(n \log n)$ on average, but in the worst case, it can grow as much as $\mathcal{O}(n^2)$.

Therefore, the final time complexity of Aldous-Broder's algorithm is:

$$\mathcal{O}(n \log n) \quad \text{on average, or} \quad \mathcal{O}(n^2) \quad \text{in the worst case.}$$

5.1.2 Wilson's Algorithm

Some assumptions:

- Visited and Unvisited cells are `sets`.
- Path is a `list`.

Choosing a random cell from the grid takes $\mathcal{O}(1)$ time with direct index access. Initializing a path (list) and appending to it are $\mathcal{O}(1)$ operations on average.

Let's now analyse the inner loop:

```
1: [...]
2: while Current is not visited do                          ▷ O(1): checking membership in a set
3:     Find all unvisited neighbours of Current             ▷ O(1): max 4 neighbours, constant checks
4:     Choose a random neighbour (call it Next)             ▷ O(1): random from constant-sized list
5:     if Next is already in Path then                                ▷ O(n): linear search in list
6:         Erase the loop                                   ▷ O(n): remove up to n elements
7:     else
8:         Append Next to Path                                                      ▷ O(1)
9:     end if
10: end while
11: [...]
```

Then, the cost per iteration of the inner loop is:

$$\mathcal{O}(1) + \mathcal{O}(1) + \mathcal{O}(n) + \mathcal{O}(n) + \mathcal{O}(1) = \mathcal{O}(n)$$

Since the loop can run up to n times (in the worst case visiting every cell), the total cost of the inner loop is:

$$n \times \mathcal{O}(n) = \mathcal{O}(n^2)$$

1: Choose a random cell from the grid and mark it as *End* ▷ $\mathcal{O}(n)$: convert set to list + random choice
2: Mark *End* as visited ▷ $\mathcal{O}(1)$
3: **while** there are unvisited cells **do**
4: Choose a random unvisited cell ▷ $\mathcal{O}(n)$
5: Initialize an empty list called *Path*
6: Append *Current* to *Path* ▷ $\mathcal{O}(1)$
7: Inner Loop ▷ $\mathcal{O}(n^2)$
8: **for** each pair of consecutive cells in Path **do**
9: Remove the wall between them ▷ $\mathcal{O}(1)$
10: Mark the cells as visited
11: **end for**
12: **end while**

Although the inner loop is nested within the outer loop, which might suggest a total time complexity of $n \times \mathcal{O}(n^2) = \mathcal{O}(n^3)$, this is not the case in practice. This estimate assumes that every loop iteration processes a distinct set of n elements, which is overly pessimistic.

In Wilson's algorithm, only the first few random walks are potentially long. As the algorithm proceeds and more cells become part of the visited tree, the probability of a new walk quickly connecting to the tree increases. This significantly reduces the expected length of subsequent walks.

Additionally, after each completed walk, the number of unvisited cells decreases, often by more than one cell, since entire paths are added at once. This accelerates convergence and further reduces the number and length of walks required.

Therefore, while individual walks may occasionally approach $\mathcal{O}(n)$ cost, the expected cost per walk is $\mathcal{O}(n)$, and with at most n walks, the total expected time complexity of Wilson's algorithm is: $\mathcal{O}(n^2)$

5.1.3 Randomized Prim

Some assumptions:

- Visited and Unvisited cells are `sets`
- Frontier is a `list`.

Choose a random cell from the grid (call it *Start*)
Mark *Start* as visited ▷ $\mathcal{O}(1)$
Add unvisited neighbors of *Start* to a set called *Frontier* ▷ $\mathcal{O}(1)$: max 4 neighbours + check if in visited set + append elements (max 4)
while *Frontier* is not empty **do**
    Choose a random cell from *Frontier* (call it *Current*) ▷ $\mathcal{O}(1)$
    Let *VisitedNeighbors* be the visited neighbors of *Current* ▷ $\mathcal{O}(1)$
    Choose one *VisitedNeighbor* at random ▷ $\mathcal{O}(1)$
    Remove the wall between *Current* and the chosen *VisitedNeighbor* ▷ $\mathcal{O}(1)$
    Mark *Current* as visited ▷ $\mathcal{O}(1)$
    Add all unvisited neighbors of *Current* to *Frontier* ▷ $\mathcal{O}(1)$: max 3 elements to add
    Remove *Current* from *Frontier* ▷ $\mathcal{O}(1)$
end while

Computing the final time complexity of the Randomized Prim's algorithm requires analyzing the cost of a single iteration of the `while` loop and multiplying it by the total number of cells in the grid, as the process is executed one cell at a time.

Each iteration has a complexity of $\mathcal{O}(1)$, since all operations (such as selecting a random element from the frontier, updating visited cells, and modifying data structures) are constant-time operations, on average, when the frontier is implemented as a list.

$$n \times \mathcal{O}(1) = \mathcal{O}(n)$$

Note: The time complexity of Randomized Prim's algorithm depends on how the *Frontier* is implemented. In this analysis, we assume it is a `list`, which allows constant-time random access. If instead it were implemented as a `set`, selecting a random element would require converting the set into a list, leading to $\mathcal{O}(n)$ operations per iteration. In that case, the overall complexity would be $\mathcal{O}(n^2)$.

5.1.4 Backtracking Algorithm

Some assumptions:

- Visited and Unvisited cells are `sets`
- Stack is a `list`.

Choose a random starting cell and mark it as visited ▷ $\mathcal{O}(n)$: convert set to list + random
Initialize an empty stack
Push the starting cell onto the stack ▷ $\mathcal{O}(1)$
while stack is not empty **do**
    Let *current* be the top cell on the stack ▷ $\mathcal{O}(1)$: append an element to a list
    Find all unvisited neighbors of *current* ▷ $\mathcal{O}(1)$: max 4 neighbours, constant checks
    if there are unvisited neighbors **then**
        Choose one neighbour randomly (call it *next*) ▷ $\mathcal{O}(1)$: random choice from constant-sized list or set
        Remove the wall between *current* and *next* ▷ $\mathcal{O}(1)$
        Mark *next* as visited ▷ $\mathcal{O}(1)$
        Push *next* onto the stack ▷ $\mathcal{O}(1)$
    else
        Pop the top cell from the stack ▷ $\mathcal{O}(1)$
    end if
end while

Computing then final time complexity of Recursive Backtracking algorithm, we have to check which is the computational cost of one iteration of the while loop and multoply it by n cells in the grid. One iteration complexity is $\mathcal{O}(1)$, hence

$$n \times \mathcal{O}(1) = \mathcal{O}(n)$$

5.1.5 Eller

Assumptions

- Cells in a row are stored in different `sets`.

Merging Set ID Loop

```
[...]
for each pair of adjacent cells in the row do                    ▷ At most c − 1 pairs
    if the cells belong to different sets and random chance allows then
        Merge the sets: assign the same set ID                   ▷ O(1)
        Remove the wall between the two cells                    ▷ O(1)
    end if
end for
[...]
```

Merging Set ID Loop. Each operation inside the loop takes constant time, and since there are at most $c - 1$ adjacent cell pairs in a row with c columns, the time complexity of this loop is $\mathcal{O}(c)$.

Vertical Connection Loop

```
[...]
for each unique set in the row do                  ▷ Let the number of sets be k ≤ c
    Randomly select one or more cells from the set ▷ O(c/k): sampling dominates list conversion
    for each selected cell do
        Remove the wall between the current cell and the cell below   ▷ O(1)
        Assign the cell below to the same set                        ▷ O(1)
    end for
end for
[...]
```

Vertical Connection Loop. Assuming the set count is $k \leq c$, the average number of elements per set is c/k. Therefore, the loop's overall complexity is$\mathcal{O}(k) \times \mathcal{O}(c/k) = \mathcal{O}(c)$.

Final Merge Loop

```
[...]
for each pair of adjacent cells in the final row do              ▷ At most c − 1 pairs
    if they belong to different sets then
        Merge the sets and remove the wall                       ▷ O(1)
    end if
end for
[...]
```

Final Merge Loop. This loop also operates on at most $c - 1$ pairs with constant-time operations, yielding a time complexity of $\mathcal{O}(c)$.

1: Start from the top-left cell (denoted as *Start*)
2: **for** each row in the grid **except the last one do** ▷ Repeated for $r - 1$ rows
3: Assign a unique set ID to each cell that does not yet have one ▷ $\mathcal{O}(1)$
4: **Merging Set ID Loop** ▷ $\mathcal{O}(c)$
5: **Vertical Connection Loop** ▷ $\mathcal{O}(c)$
6: **end for**
7: **Final Merge Loop** ▷ $\mathcal{O}(c)$

Overall Time Complexity. Combining all components, the total time complexity of Eller's algorithm is:

$$(r - 1) \times \mathcal{O}(c) + \mathcal{O}(c) = \mathcal{O}(rc) = \mathcal{O}(n)$$

which is linear in the number of cells n.

Empirical Time Complexity

Following the discussion of theoretical time complexity, we present an empirical analysis of maze generation performance. The results are based on experiments where mazes of sizes ranging from 2×2 to 100×100 were generated using the implemented algorithm.

For each dimension, the generation time was measured over 50 iterations, and the average time was computed to reduce the impact of random fluctuations and provide a more reliable estimate.

The resulting plot illustrates the actual runtime behavior of the implementation. While this does not represent the theoretical time complexity, it provides valuable insight into how closely the empirical performance aligns with the expected growth rate. Deviations may arise due to constant factors, implementation details, or system-specific overhead.

This comparison helps validate whether the implementation behaves as expected in practice and highlights any discrepancies between theory and real-world execution.

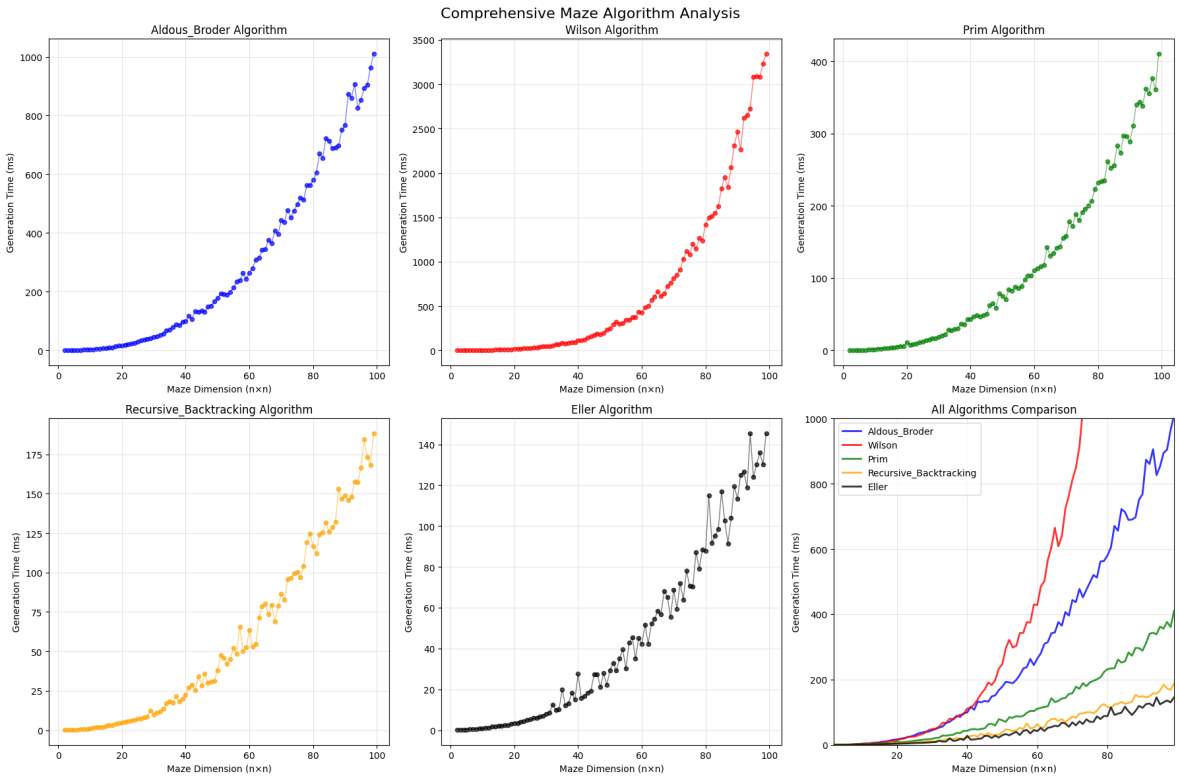


Figure 13: Empirical analysis of the mean maze generation time for grid sizes ranging from 2×2 to 100×100, averaged over 50 iterations per dimension.

5.2 Space Complexity Analysis

For maze generation algorithms, the space complexity depends primarily on the number of cells in the grid, denoted as n = rows $\times$ columns. Regardless of the specific algorithm used, each cell must be tracked in some form.

In general, the following data structures contribute to the space complexity:

- **Visited tracking:** To ensure that no cell is visited more than once (or to detect and prevent loops), each algorithm maintains a data structure such as a boolean flag, a set, or a list to track visited cells. This mechanism requires $\mathcal{O}(n)$ additional space, where n is the total number of cells in the maze.
- **Visited tracking**: To ensure no cell is visited more than once (or to detect loops), a set or boolean flag per cell is maintained, or stored in a set or list. This also takes $\mathcal{O}(n)$ space.
- **Auxiliary structures**: Depending on the algorithm, temporary data like a path (in Wilson's algorithm), a frontier (in Prim's), or a stack (in Recursive Backtracking) may be used. However, their size is at most proportional to the number of cells and never exceeds $\mathcal{O}(n)$.

Since all necessary data structures grow at most linearly with the number of cells, and no structure requires more than n entries, the overall space complexity for these algorithms is $\mathcal{O}(n)$. This result holds for all standard maze generation algorithms, including Aldous-Broder, Wilson's, Randomized Prim's, Recursive Backtracking.

Different is the case of Eller's algorithm. Despite space complexity remains $\mathcal{O}(n)$, unlike other algorithms, Eller processes the maze row by row and only needs to keep track of the current and

partially the next row. Therefore, its memory requirement is $\mathcal{O}(c)$, where c is the number of columns. However, since $c \leq n$, its space complexity still falls under $\mathcal{O}(n)$ in asymptotic terms, making it one of the most memory-efficient maze generation algorithms.

6 Comprehensive Analysis: Maze Generation Algorithms and A* Performance

6.1 Maze Inherent Metrics Analysis

This analysis examines the performance and structural characteristics of the five maze generation algorithms, evaluating 1000 instances of 100×100 mazes, analyzing dead ends, junctions, generation time, total nodes, and efficiency scores (see Section 2.3.1).

6.1.1 Raw Data Summary

Table 1: Performance Metrics for Maze Generation Algorithms

Algorithm	Dead Ends	Junctions	Time (ms)	Corridors	Effic. (%)	Complex. Ratio
Aldous-Broder	2931.58	2575.63	700.37	4492.79	7.86	1.1382
Wilson	2931.61	2574.85	2609.67	4493.54	2.11	1.1386
Prim	3560.85	2945.26	305.75	3493.89	21.28	1.2090
Recursive Back.	1002.02	982.95	125.11	8015.03	15.87	1.0194
Eller	2991.53	2574.10	100.66	4434.37	55.29	1.1622

Ranking

a. The complexity ratio ranking shows perfectly the generation tiers:

Table 2: Structural Complexity Analysis

Algorithm	Dead Ends	Junctions	Complexity Ratio
Prim	3560.85	2945.263	1.2090
Eller	2991.86	2574.10	1.1622
Aldous-Broder	2931.58	2575.63	1.1386
Wilson	2931.61	2574.85	1.1382
Recursive Backtracking	1002.02	982.95	1.0194

b. Different is the ranking result based on efficiency score, which ranks algorithms considering how rapidly junctions and dead ends are generated:

Table 3: Algorithm Performance Ranking

Rank	Algorithm	Efficiency
1	Eller	55.29
2	Prim	21.28
3	Recursive Backtracking	15.87
4	Aldous-Broder	7.86
5	Wilson	2.11

Looking at Table 2 and Table 3, we can make a grounded analysis of how the algorithms differ in characteristics and divide them into three categories:

1. **High Complexity, High Efficiency**: Prim, Eller

2. **Moderate Complexity, Low Efficiency**: Aldous–Broder, Wilson

3. **Low Complexity, Moderate Efficiency**: Recursive Backtracking

6.1.2 Generation Analysis

Let us now analyze in depth the average data presented in the previous two tables.

Structural Complexity:

Prim's Algorithm emerged as the clear leader in generating structurally complex mazes, producing an average of 3560.85 dead ends and 2946.26 junctions, yielding a complexity ratio of 1.209. This makes Prim's mazes the most intricate and challenging to navigate, since for every intersection the algorithm generates the highest average number of dead ends, and therefore the highest number of possible choices (or wrong choices). In contrast, Recursive Backtracking generated significantly simpler maze structures with only 1002.02 dead ends and 982.95 junctions, resulting in a complexity ratio of just 1.0194. This indicates that the mazes generated by this algorithm, on average, have the lowest number of possible wrong choices. Eller's algorithm ranks second, although the difference from Prim's is relatively large. Nevertheless, its performance remains valuable.

In terms of structural complexity, we can thus confirm the quick analysis made at the end of the data presentation subsection.

Performance Analysis:

The efficiency rankings revealed that Eller's Algorithm dominated with an exceptional efficiency score of 55.29, significantly outperforming all other algorithms. Prim's Algorithm, despite its top position in structural complexity, ranks second with a score of 21.28. Although this is less than half of Eller's score, its large gap over the third-ranked algorithm confirms the high performance of both Eller and Prim in terms of generation complexity and time.

By contrast, Wilson's Algorithm showed the poorest performance despite producing good-quality mazes in terms of structural complexity, achieving only 1.58 efficiency points due to its extremely slow generation time.

Complexity–Performance Paradox:

While one might expect simpler mazes to be generated faster, the data reveals otherwise. Recursive Backtracking's simple mazes required 125.11 milliseconds to generate, while Eller's significantly more complex mazes were completed in just 100.66 milliseconds. This paradox highlights a crucial distinction between structural complexity (the resulting maze characteristics) and algorithmic complexity (the efficiency of the generation process). Eller's Algorithm demonstrates that superior algorithmic design can overcome the apparent burden of creating more complex structures.

Conclusion:

This comparative analysis reveals that, from an inherent data perspective, Eller's Algorithm provides the optimal balance of maze quality and generation speed for 100×100 mazes. Although Eller ranks second in structural complexity, it still produces reasonably complex mazes that are much closer in structure to those generated by high-complexity algorithms than to the simpler mazes generated by Recursive Backtracking. Thus, we demonstrate that algorithmic efficiency is not solely determined by maze complexity, as evidenced by Eller's superior performance despite generating structurally complex mazes. For practical applications requiring maze generation, Eller's Algorithm represents the clear choice for optimal performance and complexity.

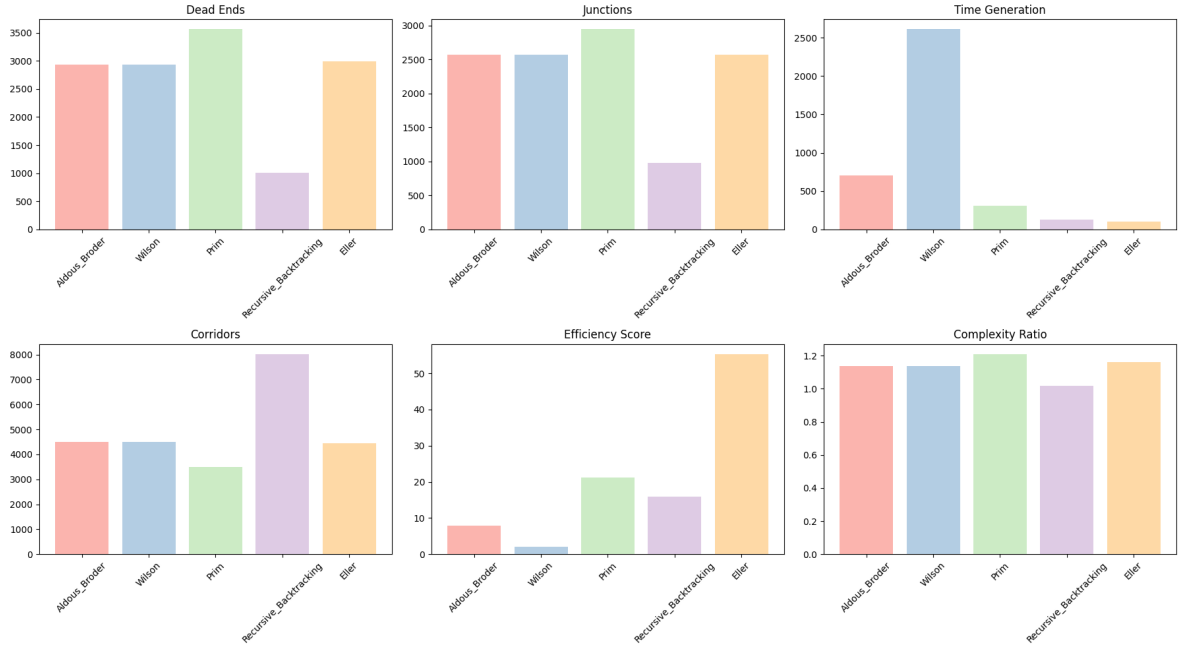


Figure 14: Maze Inherent Metrics Analysis plots

6.2 Solver-Based Metrics Analysis

In this section we are going to analyze how A* pathfinding algorithm performs on 1000 100×100 cell mazes. We decided to include Agent-Based metrics since Maze generation algorithms influence the structure of generated mazes, which in turn affects the performance of pathfinding algorithms (see Section 2.3.2).

6.2.1 Raw Data Summary

Table 4: A* Average Performance Metrics by Maze Generation Algorithm

Algorithm	**Steps**	**% Cells Visited**	**Time (ms)**	**Visited Nodes**	**Steps/s**
Aldous-Broder	6650.15	66.50	107.74	3660.54	61.72
Wilson	6651.57	66.52	107.07	3663.58	62.12
Prim	2482.75	24.83	46.33	1609.61	53.59
Recursive Backtracking	6640.59	66.41	116.17	1298.39	57.16
Eller	3247.28	32.47	40.73	1798.99	79.73

Table 5: Extended Performance Metrics

Algorithm	**Steps/Node Visited**	**Nodes Coverage**	**Corridors Visited (%)**
Aldous-Broder	1.817	66.46	44.96
Wilson	1.816	66.47	44.92
Prim	1.54	24.74	35.17
Recursive Backtracking	5.11	65.42	80.45
Eller	1.80	32.33	44.60

Ranking

a. The solution time metric ranks the algorithms in the following order (recall that in agent metrics, in general, higher the solution time, higher the difficulty in solving the maze or the slower is in finding a new path due to long branches):

Table 6: Solution Time Ranking (Ranked by difficulty/longness in solution)

Rank	Algorithm	Solution Time (ms)	Corridors (%)
1	Recursive Backtracking	116.17	80.45
2	Aldous-Broder	107.74	44.96
3	Wilson	107.07	44.92
4	Prim	46.33	35.17
5	Eller	40.73	44.60

b. Looking at the Nodes Coverage, similar results have been archived:

Table 7: Nodes Coverage Analysis

Rank	Algorithm	Nodes Coverage
1	Wilson	66.47
2	Aldous-Broder	65.45
3	Recursive Backtracking	65.42
4	Eller	32.33
5	Prim	24.74

Considering the ranks in Table 6 and Table 7, we can intuitively conduct a brief analysis, yielding the following results:

1. **High Solving Time, Moderate % Corridors**: Wilson, Aldous-Broder
2. **High Solving Time, High % Corridors**: Recursive Backtracking
3. **Low Solving Time, Low % Corridors**: Eller, Prim

6.2.2 Performance Analysis

In the generation analysis, we ranked the algorithms based on the complexity ratio and efficiency score. In this case, however, we use node coverage, solution time, and the percentage of corridors visited during the maze exploration. Although there is a clear division between mazes that require more time to be solved and those that are solved quickly, Recursive Backtracking once again presents

unexpected results. To rank the algorithms in terms of how challenging their mazes are to solve, we must analyze how each generated maze was handled by the A* agent.

It's important to note that A* is a specific, predetermined algorithm that may have inherent biases, it might perform better on mazes with certain structures or patterns. As a result, different solving agents could potentially navigate the same mazes in different ways and produce different performance metrics. Therefore, the analysis in this case is agent-specific.

In the conclusion section, we will propose future directions for studying maze generation algorithms. This includes incorporating different types of solving agents to better understand how various generation methods interact with a range of solving strategies.

Finally, to identify the algorithm that generates the most challenging and unpredictable mazes, we sum the ranks of each algorithm across all performance metrics. In this context, higher values, such as more steps, longer solving time, or more visited nodes, indicate a more difficult maze for the A* agent. Therefore, the algorithm receiving the lowest score (i.e., consistently ranked near the "worst" performer) is considered the "best" at creating hard-to-solve mazes.

Aldous-Broder's Algorithm

The Aldous-Broder algorithm presents one of the most challenging scenarios for A* pathfinding. The agent required 6,650 steps, the second highest among all algorithms (really close to the highest value), even surpassing Recursive Backtracking, which typically involves more steps due to backtracking. Additionally, Aldous-Broder exhibited the second slowest performance in both absolute time (107.74) and moderate high number of steps per second (60.78), indicating computational inefficiency.
This difficulty is further reflected in Nodes Coverage. Aldous-Broder's high score suggests that the agent needs to explore more than 66% of the entire grid to find the correct path. Notably, the mean number of steps needed for visiting a new node (1.82) is among the highest, implying that the Aldous-Broder algorithm could create a more unpredictable pattern.

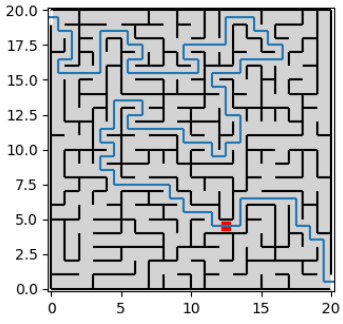


Figure 15: 20 × 20 solved Aldous-Broder maze.

Wilson's Algorithm

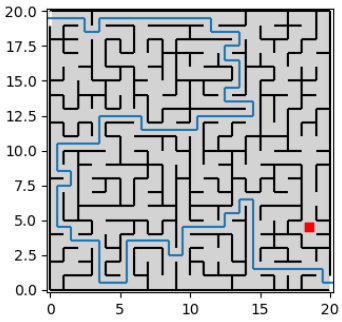


Figure 16: 20 × 20 solved Wilson maze.

As the previoius algorithm, Wilson's algorithm presents a challenging computational challenge for A* pathfinding. It requires the highest number of steps (6651.57) and visited nodes (3663.58), values comparable to Aldous-Broder, and it achieves a slightly faster solving time (107.07 ms vs. 1107.74 ms). This improvement is reflected in its higher value in steps per second (62.12 vs 61.72), indicating more efficiency node exploration per unit time.
In summary, Wilson's algorithm is a viable alternative to Aldous-Broder, as their maze structures yield similar pathfinding difficulty. However, Wilson's marginally faster solving speed suggests its mazes are slightly less challenging for A* due to subtle biases in its random walk process.

Randomized Prim's Algorithm

Prim's algorithm demonstrates strong compatibility with A*, producing mazes that are the easiest to solve among the tested algorithms. The data reveals striking differences:

- Second-fastest solving time (46.33 ms), less than half that of Aldous-Broder or Wilson.
- Fewest steps (2482.75) and lowest node visitation rate (24.83%), indicating straightforward paths.
- Lowest number of visited nodes, confirming rapid solution convergence.

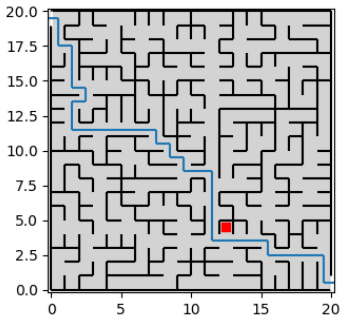


Figure 17: 20 × 20 solved Randomized Prim maze.

These results suggest that Prim's mazes contain clear, direct pathways, reducing the need for backtracking. Although Prim's generation was classified as having the highest complexity ratio in the inherent analysis, for the A* agent, Prim's mazes are computationally efficient and appear to produce the simplest paths.

Recursive Backtracking's Algorithm

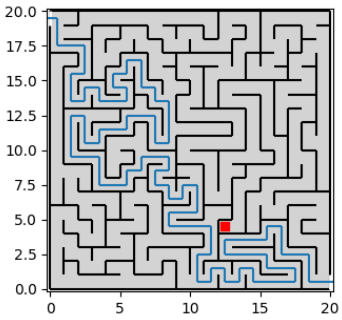


Figure 18: 20 × 20 solved Recursive Backtracking maze.

Recursive Backtracking presents the most controversial performance profile among the tested algorithms. At first glance, the data suggests extreme difficulty: the agent required 6640.59 steps and visited 66.41% of all nodes—metrics comparable to Aldous-Broder and Wilson. Its unusually high ratio of steps per visited node (5.12, nearly three times higher than the others) further implies severe inefficiency, as though the algorithm struggles to discover new nodes. This inefficiency is reflected in the extremely high proportion of corridor visits: more than 80% of the visited cells were corridors.

Yet this interpretation obscures a subtler reality. Recursive Backtracking's bias toward long, winding corridors (evident from its generation phase) forces the agent to spend excessive time traversing deep paths before reaching dead ends. Paradoxically, the algorithm generates relatively few dead ends (as confirmed by the generation analysis), but those that do exist are buried behind elongated corridors. Thus, while the maze topology is not inherently complex, its corridor-dominated structure induces temporal inefficiency.

In essence, Recursive Backtracking's poor performance metrics stem not from navigational complexity but from structural monotony. The algorithm is slow not because the solution is difficult to identify, but because the paths leading to dead ends are disproportionately long.

Eller's Algorithm

Eller's algorithm delivers strong but unsurprising performance, ranking among the easiest mazes for A* to solve. While it outperforms Prim's algorithm in raw metrics, achieving a 32.33% node visitation rate (vs. Prim's 24.74%) and the fastest solving time (40.73 ms), this efficiency comes at the cost of reduced complexity.

The algorithm's row-by-row generation introduces a subtle horizontal bias, creating balanced but predictable pathways. This is reflected in its record-low solution time, indicating that A* navigates new nodes almost effortlessly.

Ultimately, Eller's mazes are optimized for solvability: their controlled structure minimizes dead-ends and backtracking, making them computationally trivial. For applications prioritizing pathfinding speed, this is ideal; for testing robust exploration, they lack the organic challenge of algorithms like Aldous-Broder.

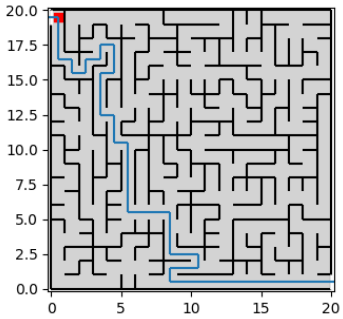


Figure 19: 20×20 solved Eller maze.

Table 8: Final Algorithm Ranking Based on Sum of Metric Ranks

Algorithm	**Steps**	**Time**	**Nodes Visited.**	**Nodes Coverage**	**Final Rank**
Wilson	1	3	1	1	1
Aldous-Broder	2	2	2	2	2
Recursive Back.	3	1	5	3	3
Eller	4	5	3	4	4
Prim	5	4	4	5	5

Ranking Method: Each metric is ranked from 1 (best) to 5 (worst). The total score is the sum of the ranks. The final ranking is based on the lowest total score.

Final Ranking

The final ranking (Table 8 is determined by summing the individual ranks each algorithm received across all performance metrics. The algorithm with the lowest total score is ranked highest, as lower values indicate better performance in each metric.

This approach provides a simple visual summary of overall performance by aggregating the ranks from each metric. It does not assign different weights to metrics, making it an intuitive but non-statistical way to compare algorithm efficiency at a glance.

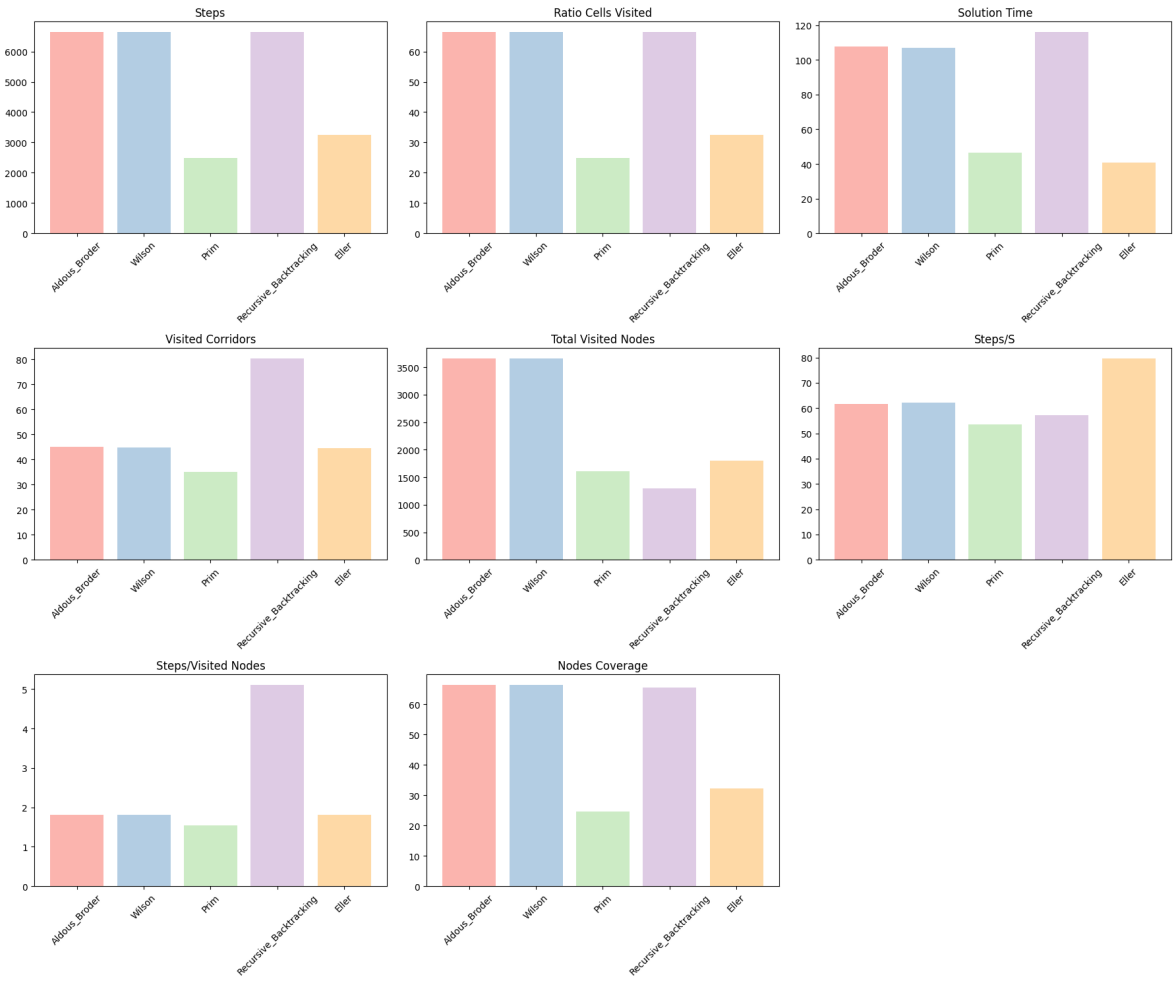


Figure 20: Maze Agent Metrics Analysis plots

Analysis Conclusions

Looking at both maze-inherent properties and agent-based performance, we can draw some insightful conclusions about the behavior of different maze generation algorithms.

While the complexity ratio suggested that Randomized Prim was the most randomized algorithm, due to its high number of dead ends relative to intersections, the agent-based analysis revealed a different story. Despite its structural complexity, Prim's mazes follow a somewhat predictable pattern, allowing the A* agent to find a solution quickly.

Wilson's algorithm and Aldous-Broder, ranked 1st and 2nd respectively in overall ranking, both showed relatively good complexity ratios, indicating a high degree of path randomization. This was confirmed by the agent-based analysis: Wilson proved to be the most difficult maze to solve, while Aldous-Broder came in second. However, both algorithms are the slowest to generate, suggesting a trade-off between maze complexity and generation speed.

Eller's algorithm stands out as the most enigmatic of the group. It generates mazes very quickly

and maintains a good complexity ratio, yet it results in very low difficulty for the A* agent. This suggests that while the mazes appear complex, they are easier to solve, possibly due to underlying structural patterns.

The most contrasting algorithm is Recursive Backtracking. It scores the lowest in complexity, yet it ranks third in the agent-based analysis. This apparent contradiction stems from its inherent bias: the long, winding paths and deep dead ends force the agent to backtrack extensively, increasing solution time. As the complexity analysis suggests, it produces mazes with few junctions and decision points, making them structurally less engaging, despite the challenge they pose to the agent.

Final Summary:

- Aldous-Broder and Wilson are the worst nightmares for a player: high complexity, no discernible pattern, and many decision points make it the most challenging maze to solve.
- Prim and Eller are ideal for games with predictable patterns that are relatively easy to solve.
- Recursive Backtracking generates the least complex mazes, which may feel repetitive or frustrating for players seeking dynamic exploration, though it allows for deep exploration of multiple branches.

7 Conclusion

In summary, each maze generation algorithm exhibits unique characteristics that influence both the structure of the maze and the difficulty of solving it. From the comparative analysis, Wilson and Aldous-Broder emerged as the most challenging mazes for pathfinding agents, offering high complexity, no discernible patterns, and numerous decision points, making it the ideal choice for applications where unpredictability and difficulty are desired.

On the other hand, Prim and Eller algorithms produce mazes that are structurally more predictable and easier to solve, making them suitable for games or applications where a smoother, more guided player experience is preferred.

Finally, Recursive Backtracking, while generating the least complex mazes, proved to be deceptively challenging due to its long, winding paths and frequent dead ends. While this can lead to repetitive gameplay, it also allows for deep exploration of multiple branches, a feature that could be leveraged in procedural content generation or puzzle design.

7.1 Future Studies

To build on this analysis, future work could explore several promising directions. One area of interest is the use of multiple agents in pathfinding and maze-solving tasks, which could lead to more efficient exploration and better insights into maze complexity. Additionally, investigating new or hybrid maze generation algorithms could help improve both generation speed and structural complexity.

Another valuable direction is studying how maze properties affect player experience in real-time, particularly by adapting maze generation dynamically based on player performance. This could enable the creation of personalized, adaptive gameplay experiences where the challenge level evolves in response to the player's skill and progress.

Finally, combining elements from different algorithms, such as the randomness of Wilson or Aldous-Broder and the efficiency of Prim, could lead to the development of new hybrid algorithms that balance complexity and performance. Such an approach has the potential to significantly enhance both maze quality and computational efficiency, making it a compelling area for further research.

References

Disclaimer: This report was partially drafted using ChatGPT to refine the grammar and syntax of several sections. Additionally, portions of the code for Eller's algorithm were partially generated with the assistance of ChatGPT to resolve coding difficulties.

1. Diestel, R. (2017). *Graph Theory* (5th ed.). Springer. ISBN: 978-3-662-53621-6.
2. Lovász, L. (1993). *Random Walks on Graphs: A Survey.* In Combinatorics, Paul Erdős is Eighty (Vol. 2, pp. 1–46).
3. Graph thoery notions: `https://en.wikipedia.org/wiki/Graph_theory#:~:text=In%20mathematics%20and%20computer%20science,arcs%2C%20links%20or%20lines)`
4. Maze generation algorithms: `https://en.wikipedia.org/wiki/Maze_generation_algorithm`
5. Graph Theory overview: `https://amslaurea.unibo.it/id/eprint/13824/1/TESI%20tommi.pdf`
6. Algorithms Pseudocodes: `https://weblog.jamisbuck.org/2011/1/17/maze-generation-aldous-broder-algorithm`
7. A-star code: `https://github.com/khaledkamr/Maze-solver-using-A-star/blob/main/Maze%20solver.py`

8 Appendix A

In paper presentation (Section 1), we presented what a maze is and how it is structured, while in Mathematical Modelling (Section 2) we introduced the concept of Graph and its Tree representation. Here we go slightly in depth to show how a maze can be represented as a Tree.

8.1 How structure mazes as Trees?

To structure a maze as a tree we need to rename its elements:

- **Node**: a node is a position cell
- **Root**: the root is the initial starting point of a maze, i.e, the point form which the maze starts to be generated. Note that the root is a node, but one and only one node can be the root
- **Links**: a link is a line which connects a node to an other node (nodes can have up-direction links (Child), down-direction links (Child), or both)

Some examples:

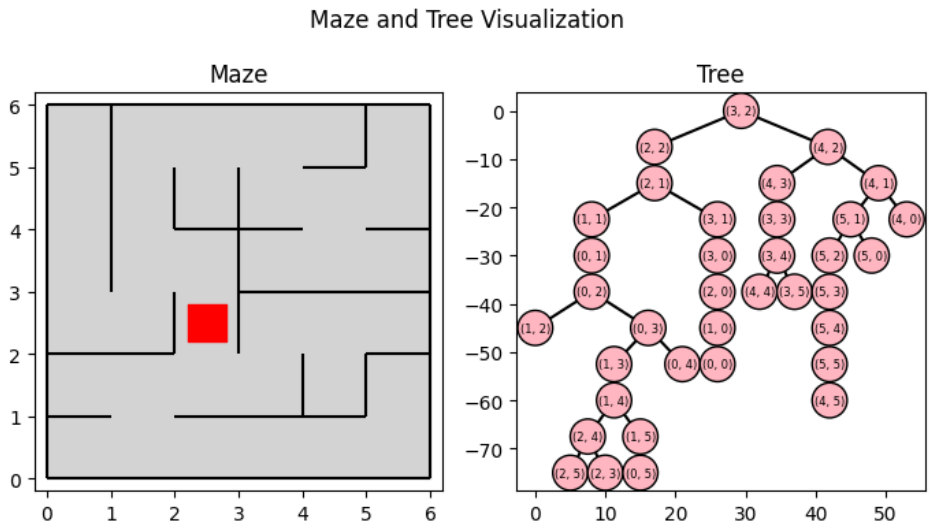


Figure 21: A 6x6 maze and its tree representation generated using Aldous-Broder algorithm

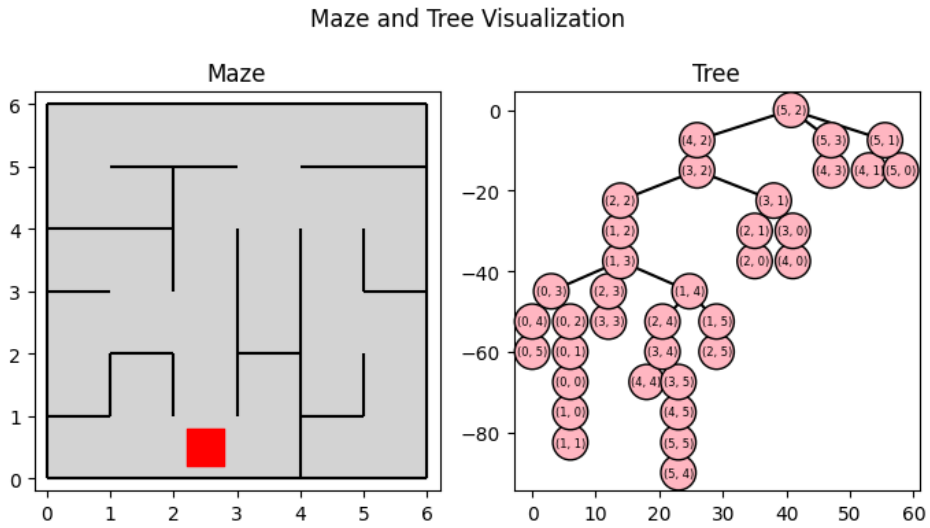


Figure 22: A 6x6 maze and its tree representation generated using Wilson algorithm

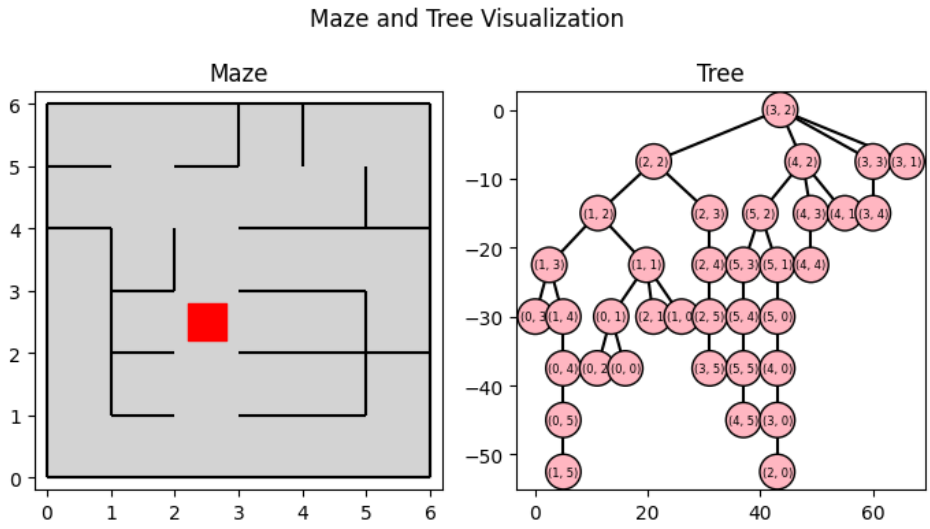


Figure 23: A 6x6 maze and its tree representation generated using Prim algorithm

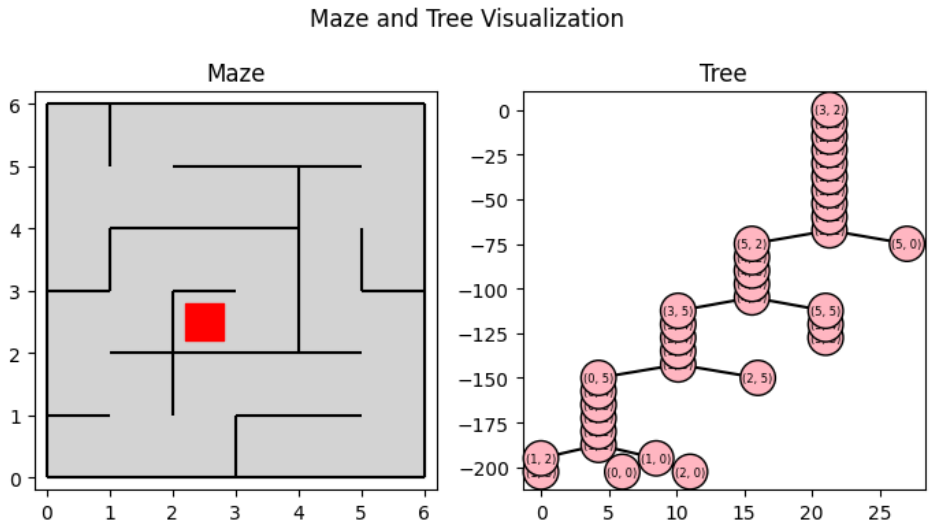


Figure 24: A 6x6 maze and its tree representation generated using Recursive Backtracking algorithm

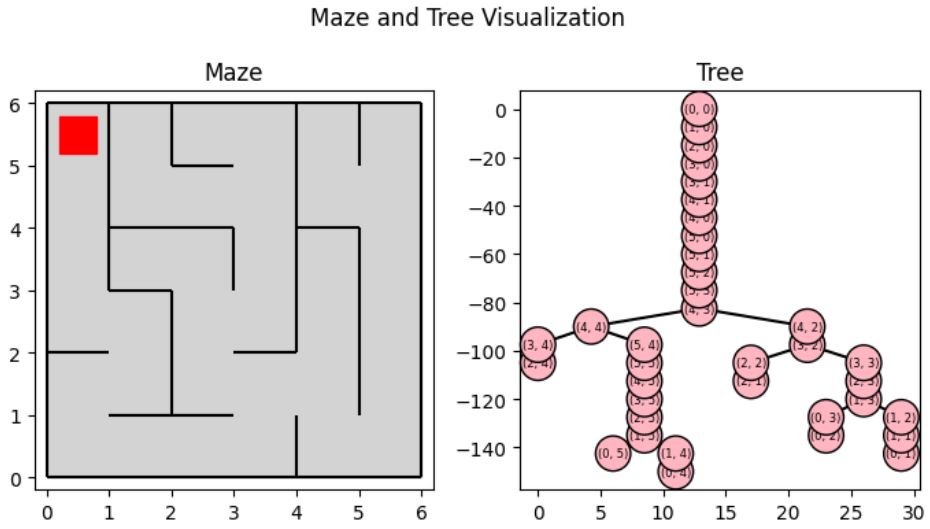


Figure 25: A 6x6 maze and its tree representation generated using Eller algorithm

9 Appendix B

In this section, all auxiliary functions are presented, organized by algorithm. Note that here there are only functions that are algorithm specific.

9.1 Aldous-Broder

```
class Aldous_Broder(MazeInitializer):
    """Contains all function about Aldous-Broder generation algorithm
    Takes MazeInitializer as parent from which obtains maze Initialization functions and parametes

    Attributes:
        rows (int): Number of rows in the maze.
        cols (int): Number of columns in the maze.
    """

    def __init__(self, rows, cols, step_by_step = None, visualizer = None,  plot_size = (3.5, 3.5)):
        """Gets all parameters from the parent
        Args:
            step_by_step (bool): visualize step by step the generation of the maze. Default is None.
            visualizer (class): class of visualizing functions
            """
        super().__init__(rows, cols, step_by_step, visualizer,  plot_size)


    def next_cell(self, current_cell):
        """Chooses a cell from the grid.
        Args:
            current_cell (tuple): current cell
        Return:
            next_cell (tuple): coodinates of the next cell
        """
        neighbours = self.neighbourhood(current_cell, self.grid) # retrieves neighbours present in the given set or list
        next_cell = random.choice(neighbours) # randomly chooses the cell

        if self.step_by_step:
            self.visualizer.represent_cell(self.rows, next_cell, 'lightgrey') # show the cell in the plot
            self.visualizer.show_step(self.plot_size)

        return next_cell

    def place_wall(self, position, previous_position):
        """Places the wall where it finds already visited neighbours
        Args:
            current_path (list): list of all mined path
        """

        neighbours = self.neighbourhood(position, self.grid) # define neighbourhood
        random.shuffle(neighbours) # shuffle the neighbours to not start always the same direction

        for neigh in neighbours:
            if neigh != previous_position: # leaves opened the passage
                if neigh in self.visited_cells:
                    move = self.retrieve_move(position, neigh) # retrives the necessaty move to go from position to the neighbour
                    self.maze_map = self.update_maze_map(position, neigh, move, self.maze_map) # update the maze_map

```

```
50          if self.step_by_step:
51              self.visualizer.show_step(self.plot_size)
```

Listing 6: Auxiliary function of Aldous-Broder generation algorithm

9.2 Wilson

```
1
2   class Wilson(MazeInitializer):
3       """Contains all function about Wilson generation algorithm
4       Takes MazeInitializer as parent from which obtains maze Initialization functions
            and parametes
5
6       Attributes:
7           rows (int): Number of rows in the maze.
8           cols (int): Number of columns in the maze.
9       """
10
11      def __init__(self, rows, cols, step_by_step = None, visualizer = None,  plot_size
            = (3.5, 3.5)):
12          """Gets all parameters from the parent
13          Args:
14              step_by_step (bool): visualize step by step the generation of the maze.
                    Default is None.
15              visualizer (class): class of visualizing functions
16              """
17          super().__init__(rows, cols, step_by_step, visualizer, plot_size) # all
                parent parameters became class parameters
18          self.path_mined = []
19
20
21
22      def next_cell(self, current_cell, available_squares):
23          """Chooses a cell from a given set or list of cells
24          Args:
25              current_cell (tuple): current cell
26              available_squares (list or set): list or set of cells from which choose
                    the next cell
27          Return:
28              next_cell (tuple): coodinated of the next cell
29          """
30          cells_choice = self.neighbourhood(current_cell, available_squares) #
                retrieves neighbours present in the given set or list
31          next_cell = random.choice(cells_choice) # randomly chooses the cell
32
33          if self.step_by_step:
34                  self.visualizer.represent_cell(self.rows, next_cell) # show the cell
                        in the plot
35                  self.visualizer.show_step(self.plot_size)
36
37          return next_cell
38
39      def cycle_hunter(self, next_cell, path):
40          """Wheter a cycle is generated, it removes it
41          Args:
42              next_cell (tuple): coordinates of the next step
43              path (list): list of all steps made until here from the path miner
                    function
44          Returns:
45              path (list): list of the path after cycle is removed"""
46
47          if next_cell in path:
48              path_index = path.index(next_cell) # retirives the index in the path
```

```
            if self.step_by_step:
                for step in path[path_index + 1:]:
                    self.visualizer.represent_cell(self.rows, step, color = 'white')
                        # recolor all elements until the root of the cycle

            path = path[:path_index] # removes all elements until the root of the
                cycle
        return path


    def path_miner(self, start):
        """Created a path from the start cell until it reaches any other cell altrady
            visited (not in the creating path)
        Arg:
            start (tuple): coordinates of the starting cell chosen using
                starting_position function
        Return:
            path (list): list of mined path
        """
        if self.step_by_step == True:
            self.visualizer.represent_cell(self.rows, start, 'lightgrey')

        next_step = self.next_cell(start, self.grid) # defines the next step
        path = [start, next_step] # initializes the path list

        while next_step not in self.visited_cells: #until we get a visited cell
            start = next_step
            next_step = self.next_cell(start, self.grid) #randomly choice a new
                position in the neighbourhood of start

            #check if cicles are present
            path = self.cycle_hunter(next_step, path) # checks and remove any cycle
            path.append(next_step) # append the root of the cycle

        if self.step_by_step == True:
            for cell in path:
                self.visualizer.represent_cell(self.rows, cell, 'lightgrey')
        return path

    def place_wall(self, current_path):
        """Places the wall where it finds already visited neighbours
        Args:
            current_path (list): list of all mined path
        """

        path = current_path
        previous_step = path[-1] #sets the previous step as the last position found
            to maintain a passage open

        path = path[:-1] # removes the last element which is the cell that connects
            the mined path with the visited cells

        for position in path[::-1]: # inverts the path to start from the origin of
            the mined path
            if position not in self.visited_cells:
                self.visited_cells.add(position)  #if the position is not visited
                    yet, mark it as visited

            neighbours = self.neighbourhood(position, self.visited_cells) # define
                neighbourhood
            random.shuffle(neighbours) # shuffle the neighbours to not start always
                the same direction (this avoids horizontal or vertical bias)

            for neigh in neighbours:
                if neigh != previous_step: # leaves opened the passage
```

```
                    move = self.retrieve_move(position, neigh) # retrives the
                        necessaty move to go from position to the neighbour
                    self.maze_map = self.update_maze_map(position, neigh, move,
                        self.maze_map) #update maze_map

            previous_step = position # it considers the last step as the previous of
                the next one

        if self.step_by_step == True:
            self.visualizer.show_step(self.plot_size)
```

Listing 7: Auxiliary function of Wilson Maze generation algorithm

9.3 Randomized Prim

```
class Prim(MazeInitializer):
    """Contains all function about Prim generation algorithm
    Takes MazeInitializer as parent from which obtains maze Initialization functions
        and parametes

    Attributes:
        rows (int): Number of rows in the maze.
        cols (int): Number of columns in the maze.
    """
    def __init__(self, rows, cols, step_by_step = None, visualizer = None,  plot_size
        = (3.5, 3.5)):
        """Gets all parameters from the parent
        Args:
            step_by_step (bool): visualize step by step the generation of the maze.
                Default is None.
            visualizer (class): class of visualizing functions
        """
        super().__init__(rows, cols, step_by_step, visualizer, plot_size)


    def place_wall(self, position, previous_position):
        """Places the wall where it finds already visited neighbours
        Args:
            position (tuple): current cell
            previous_position (tuple): cell from which maintain open the passage
        """
       #if the position is not visited yet, mark it as visited
        neighbours = self.neighbourhood(position, self.grid )# define neighbourhood
        random.shuffle(neighbours) # shuffle the neighbours to not start always the
            same direction

        for neigh in neighbours:
            if neigh != previous_position: # leaves opened the passage
                if neigh in self.visited_cells:
                    move = self.retrieve_move(position, neigh) # retrives the
                        necessaty move to go from position to the neighbour
                    self.maze_map = self.update_maze_map(position, neigh, move,
                        self.maze_map)

            if self.step_by_step:
                self.visualizer.show_step(self.plot_size)
        return

    def check_frontier(self, frontier, neighbours):
        """Retrieve which cells have to be included in frontier set
        Args:
            frontier (set): set of cells that are neighours of a visited cell
            neighbours (set or list): set or list of all neighbour of a given cell
```

```
        Returns:
            frontier (set): updated frontier
            """
        for neigh in neighbours:
            if neigh not in self.visited_cells:
                frontier.add(neigh)

                if self.step_by_step:
                    self.visualizer.represent_cell(self.rows, neigh, color = 'grey')
                    self.visualizer.show_step(self.plot_size)
        return frontier
```

Listing 8: Auxiliary function of Randomized Prim Maze generation algorithm

9.4 Recursive Backtracking

```
class Recursive_Backtracking(MazeInitializer):
    """Contains all function about Aldous-Broder generation algorithm
    Takes MazeInitializer as parent from which obtains maze Initialization functions
        and parametes

    Attributes:
        rows (int): Number of rows in the maze.
        cols (int): Number of columns in the maze.
    """
    def __init__(self, rows, cols, step_by_step = None,  visualizer = None,
        plot_size = (3.5, 3.5)):
        """Gets all parameters from the parent
        Args:
            step_by_step (bool): visualize step by step the generation of the maze.
                Default is None.
            visualizer (class): class of visualizing functions
            """
        super().__init__(rows, cols, step_by_step, visualizer,  plot_size)

    def next_cell(self, current_cell, available_squares):
        """Chooses a cell from a given set or list of cells
        Args:
            current_cell (tuple): current cell
            available_squares (list or set): list or set of cells from which choose
                the next cell
        Return:
            next_cell (tuple): coodinated of the next cell
        """
        cells_choice = self.neighbourhood(current_cell, available_squares) #
            retrieves neighbours present in the given set or list
        next_cell = random.choice(cells_choice) # randomly chooses the cell

        if self.step_by_step:
                self.visualizer.represent_cell(self.rows, next_cell) # show the cell
                    in the plot
                self.visualizer.show_step(self.plot_size)

        return next_cell

    def place_wall(self, position, previous_position):
        """Places the wall where it finds already visited neighbours
        Args:
            position (tuple): current cell
            previous_position (tuple): cell from which maintain open the passage
        """
        neighbours = self.neighbourhood(position, self.visited_cells) # define
            neighbourhood
```

```
41
42          for neigh in set(neighbours):
43              if neigh != previous_position: # leaves opened the passage
44                  if neigh in self.visited_cells:
45                      move = self.retrieve_move(position, neigh) # retrives the
                            necessaty move to go from position to the neighbour
46                      self.maze_map = self.update_maze_map(position, neigh, move,
                            self.maze_map)
47
48          if self.step_by_step:
49              self.visualizer.show_step(self.plot_size)
50          return
```

Listing 9: Auxiliary function of Recursive Backtracking Maze generation algorithm

9.5 Eller

```
1  class Eller(MazeInitializer):
2      """Contains all function about Aldous-Broder generation algorithm
3      Takes MazeInitializer as parent from which obtains maze Initialization functions
           and parametes
4
5      Attributes:
6          rows (int): Number of rows in the maze.
7          cols (int): Number of columns in the maze.
8          cell (tuple): current cell
9          next_cell (tuple): next step
10         action (str):
11             'a' = add the wall
12             'r' = remove the wall
13     """
14
15     def __init__(self, rows, cols, step_by_step = None, visualizer = None,  plot_size
           = (3.5, 3.5)):
16         """Gets all parameters from the parent
17         Args:
18             step_by_step (bool): visualize step by step the generation of the maze.
                   Default is None.
19             visualizer (class): class of visualizing functions
20         """
21         super().__init__(rows, cols, step_by_step, visualizer,  plot_size)
22         self.set_id = 1
23         self.row_sets = {}
24         self.neighbour_caches = {}
25
26     def assign_sets(self, row):
27         """Assigns unique sets to each cell in the row if the cell is not already
               assigned.
28         Arg:
29             row (int): i-th row
30         """
31         new_row = {}
32         for col in range(self.cols):
33             cell = row, col
34             if cell not in self.row_sets:
35                 new_row[cell] = self.set_id #assign a unique id to each cell
36                 self.set_id += 1
37             else:
38                 new_row[cell] = self.row_sets[cell] # assign the same id to the same
                       cell in the new dictionary
39
40             if self.step_by_step:
41                 self.visualizer.represent_cell(self.rows, cell)
```

```
        return new_row

    def place_vertical_wall(self, cell, next_cell, action:str = 'a'):
        """Place or remove vertical walls both in maze map and in plot (is
            step_by_step is True)"""

        wall_dict = self.maze_map
        #updates maze map for both adiacent cells
        wall_dict = self.update_sub_dict(wall_dict, cell, (0, 1), action)
        wall_dict = self.update_sub_dict(wall_dict, next_cell, (0, -1), action)

        self.maze_map = wall_dict

        if self.step_by_step:
            self.visualizer.wall_plot((0, 1), self.rows, self.cols, cell, action =
                action) # places the wall in the plot
            self.visualizer.show_step(self.plot_size)

    def place_horizontal_wall(self, cell, next_cell, action:str = 'a'):
        """Place or remove horizontal walls both in maze map and in plot (is
            step_by_step is True)"""
        wall_dict = self.maze_map

        wall_dict = self.update_sub_dict(wall_dict, cell, (-1, 0))
        wall_dict = self.update_sub_dict(wall_dict, next_cell, (1, 0))

        self.maze_map = wall_dict
        if self.step_by_step:
            self.visualizer.wall_plot((1, 0), self.rows, self.cols, cell) # places
                the wall in the plot
            self.visualizer.show_step(self.plot_size)

    def merge_sets(self, row):
        """Randomly merges adjacent cells horizontally.
        Arg:
            row (int): determine in which row the generation is"""

        for col in range(self.cols - 1):
            cell = (row, col)
            next_cell = (row, col + 1)

            if self.step_by_step:
                self.visualizer.represent_cell(self.rows, cell, 'grey')
                self.visualizer.represent_cell(self.rows, next_cell, 'red')

            if self.row_sets[cell] != self.row_sets[next_cell]: #checks the two ids.
                If the two ids are equal no change is needed
                if random.choice([True, False]) == True: # randomly chooses if place
                    a wall or not
                     self.merge_cells(cell, next_cell)

                else:
                    self.place_vertical_wall(cell, next_cell) # if false we do not
                        merge and we set a wall in the middle

            else:
                self.place_vertical_wall(cell, next_cell) # if the ids are equal we
                    do not merge and we add a wall in the middle

        if self.step_by_step:
            self.visualizer.represent_cell(self.rows, next_cell, 'lightgrey')

    def merge_cells(self, cell, next_cell):
        """Merges all cells with the set of cell2 into cell1's set."""
        set1 = self.row_sets[cell] # id1
```

```
        set2 = self.row_sets[next_cell] #id2
        self.row_sets[next_cell] = set1 # replace next_cell's id and merge it to cell placing the same id
        for c, set in self.row_sets.items(): # every other cell with the same set of next cell (i.e. have set2), replace its set with set1
            if set == set2:
                self.row_sets[c] = set1

    def group_cells_by_set_id(self, row_sets):
        """
        Groups cells in the current row by their set ID.

        This function takes the cells in the current row and organizes them
        into a dictionary where the keys are set IDs and the values are lists
        of cells that belong to each set.

        Arg:
            row_sets (dict): it contains all cells of a given row assignet to a set_id
        Returns:
            dict: A dictionary mapping set IDs to lists of cell coordinates.
                  Example: {1: [(2, 0), (2, 1)], 2: [(2, 2)]}
        """
        set_members = {}
        for member, set_id in row_sets.items():
            if set_id not in set_members.keys():
                set_members[set_id] = [] # create a list for every new set_id encountered

            set_members[set_id].append(member) # append all cells with the same id

        return set_members

    def _create_vertical_connections(self, row):
        """Ensures each set has at least one vertical connection."""

        set_members = self.group_cells_by_set_id(self.row_sets)

        new_row_sets = {}
        for set_id, members in set_members.items(): # for every set in the row
            if len(members) > 1: # at least one passage open
                k = random.randint(1, len(members) - 1) # number of elements have to be samoled in the set
            else:
                k = 1
            passages = random.sample(members, k) # sample which passages have to remain opened

            for cell in members:
                    r, c = cell

                    if cell not in passages: # place a wall
                        opposite_cell = (r + 1, c)
                        self.place_horizontal_wall(cell, opposite_cell)

                        new_row_sets[opposite_cell] = self.set_id # assignes a new set_id to the cell opposite to the wall
                        self.set_id += 1
                    else:
                        if row != self.rows - 1:  # if it is not the last row
                            new_row_sets[(r + 1, c)] = set_id  # assignes to the opposite passage cell the same set_id

                    if self.step_by_step:
                        self.visualizer.represent_cell(self.rows, cell, 'lightgrey')

```

```
157         if row != self.rows - 1:
158             self.row_sets = new_row_sets
159 
160     def final_merge(self):
161         """Merge cells in the last row"""
162 
163         for col in range(self.cols - 1): # define pairs of cell horizontally
164             cell = (self.rows - 1, col)
165             next_cell = (self.rows - 1, col + 1)
166 
167             if self.row_sets[cell] != self.row_sets[next_cell]: # if set_id s are
                    different, merge them and remove wall, else nothing changes
168                 self.merge_cells(cell, next_cell)
169                 self.place_vertical_wall(cell, next_cell, 'r')
170 
171             if self.step_by_step:
172                 self.visualizer.represent_cell(self.rows, cell, 'lightgrey')
173                 self.visualizer.represent_cell(self.rows, next_cell, 'red')
174 
175         if self.step_by_step:
176             self.visualizer.represent_cell(self.rows, next_cell, 'lightgrey')
```

Listing 10: Auxiliary function of Eller Maze generation algorithm